Fondamenti di Programmazione

Spark

settembre e ottobre 2026 · volume 1

Capire un programma, un passo alla volta

Un programma cambia dei dati. Per capire se lo fa nel modo giusto dobbiamo dire che cosa vogliamo ottenere, seguire le operazioni e trovare un ragionamento che valga per tutti gli ingressi ammessi. Il libro costruisce questi passaggi partendo da esempi piccoli. Servono soltanto aritmetica e algebra elementari.

1 Il percorso

Cominciamo da una richiesta, una procedura e i valori che cambiano mentre la eseguiamo: sono **specifica, algoritmo e stato** (capitolo 1).

Per scrivere proprietà precise ci serve poi la **logica**. Prima ragioniamo con frasi considerate vere o false (capitolo 2); poi impariamo a parlare di oggetti e a dire «tutti» ed «esiste» con i **quantificatori** (capitoli 3 e 4).

Torniamo quindi ai programmi. Studiamo l'effetto dei comandi (capitolo 5) e lo colleghiamo alle proprietà che valgono prima e dopo con le **triple di Hoare** (capitolo 6).

Infine affrontiamo i cicli. Gli **invarianti** spiegano che cosa si conserva; le **funzioni di terminazione** spiegano perché il calcolo finisce. Negli array useremo confini mobili per distinguere il lavoro già fatto da quello rimasto (capitoli 7–9).

2 Come leggere gli esempi

Non occorre imparare subito ogni simbolo. Leggi l'esempio e chiediti che cosa vogliamo sapere; poi torna alla definizione. Prima di un assegnamento, fermati: quale valore viene letto? Quale viene sostituito?

Nelle prove distingui ciò che sappiamo prima di un passo da ciò che dobbiamo sapere dopo. Quando scegliamo un comando o un'uguaglianza, cerchiamo anche il motivo della scelta.

Gli esercizi hanno tutti una soluzione spiegata nell'ultima parte. Prova almeno a scegliere il primo passo prima di leggerla. I passaggi «Per proseguire» sviluppano varianti secondarie e si possono rimandare a una seconda lettura.

3 Convenzioni e fonti

I naturali comprendono lo zero. Usiamo il piccolo linguaggio imperativo del corso con interi matematici, senza limiti di rappresentazione. La divisione intera e le condizioni perché un'espressione abbia valore saranno precisate prima dell'uso.

L'assegnamento $x := E$ cambia il valore di x; l'uguaglianza $x = E$ afferma una proprietà. Le *variabili di specifica* ricordano valori iniziali e restano fissate durante la prova.

Il periodo va dalla prima nota disponibile, del 19 settembre, al 31 ottobre 2026. L'ultima lezione presente è del 29 ottobre. Il volume rielabora tredici note e la trascrizione del 25 settembre. Le spiegazioni aggiunte non sono citazioni del docente; fonti e scelte sono registrate nei documenti di accompagnamento.

Indice

Capire un programma, un passo alla volta . . . 2

1 Algoritmi, specifiche e stati . . . 5

1.1 Dal problema alla specifica . . . 5
1.2 Stato e assegnamento: che cosa cambia . . . 6
1.3 Algoritmi diversi, costi diversi . . . 7
1.4 Modelli di calcolo e limiti . . . 7

2 Calcolo proposizionale: verità e conseguenza . . . 9

2.1 Formule, connettivi e interpretazioni . . . 9
2.2 Modelli, tautologie e controesempi . . . 10
2.3 Equivalenze e dimostrazioni motivate . . . 10
2.4 Implicazione e polarità delle occorrenze . . . 12

3 Logica del primo ordine: individui e formule . . . 14

3.1 Termini, predicati e quantificatori . . . 14
3.2 Portata, variabili libere e sostituzione . . . 15
3.3 Interpretazioni: i simboli ricevono un significato . . . 15
3.4 Formalizzare: tutti, alcuni, esattamente uno . . . 16

4 Semantica e dimostrazioni con i quantificatori . . . 17

4.1 Assegnamenti e valore di termini e formule . . . 17
4.2 Negazione, distribuzione e domini ristretti . . . 18
4.3 Regole di prova: individui particolari e generici . . . 19
4.4 Uguaglianza e asserzioni sugli stati . . . 20

5 Stati, espressioni e comandi . . . 22

5.1 Lo stato e la valutazione delle espressioni . . . 22
5.2 Assegnamento, sequenza e perdita di informazione . . . 23
5.3 Scelte e ripetizioni: if e while . . . 24
5.4 La semantica dei comandi e le esecuzioni senza risultato . . . 24

6 Triple di Hoare e invarianti dei cicli . . . 26

6.1 Precondizione, postcondizione e correttezza . . . 26
6.2 Conseguenza e definitezza delle espressioni . . . 27
6.3 Assegnamento: ragionare a partire dal risultato . . . 27
6.4 Sequenza: trovare l'asserzione intermedia . . . 28
6.5 Condizionale: utilizzare l'informazione della guardia . . . 28
6.6 Invariante e correttezza parziale del while . . . 29

7 Correttezza totale e funzioni di terminazione . . . 31

7.1 Una quantità naturale che diminuisce . . . 31
7.2 La regola del while in correttezza totale . . . 32
7.3 Divisione intera: completare la prova . . . 32
7.4 Somma dei primi interi: scegliere l'invariante . . . 33
7.5 Programmi annotati e proprietà non modificate . . . 34

8 Costruire un ciclo: il significato delle variabili . . . 36

8.1 Dalla postcondizione a una proprietà da conservare . . . 36
8.2 Fattoriale: ricordare il prodotto già calcolato . . . 36

8.3 MCD: trasformare i dati conservando la risposta . . . 37
8.4 Radice cubica: costruire potenze con differenze . . . 38
8.5 Fibonacci: conservare il valore che manca . . . 39

9 Array: indici, aggiornamenti e zone . . . 42

9.1 Array come funzioni sugli indici . . . 42
9.2 Aggiornamento selettivo: prima valutare, poi modificare . . . 43
9.3 Incrementare un array: un confine tra fatto e da fare . . . 43
9.4 Somma di un array: sostituire la fine con un confine mobile . . . 44
9.5 Bandiera a due colori: classificare senza perdere elementi . . . 45

Appendice A Soluzioni ragionate . . . 48

A.1 Capitolo 1 — Specifiche e stati . . . 48
A.2 Capitolo 2 — Calcolo proposizionale . . . 48
A.3 Capitolo 3 — Individui e formule . . . 48
A.4 Capitolo 4 — Semantica e quantificatori . . . 49
A.5 Capitolo 5 — Stati, espressioni e comandi . . . 49
A.6 Capitolo 6 — Triple di Hoare e invarianti . . . 49
A.7 Capitolo 7 — Correttezza totale . . . 50
A.8 Capitolo 8 — Costruire un ciclo . . . 50
A.9 Capitolo 9 — Array . . . 51

Capitolo 1

Algoritmi, specifiche e stati

In breve
Come si passa da una richiesta a un programma? Partiamo da una moltiplicazione fatta con sole somme. Distingueremo il risultato che vogliamo ottenere, le istruzioni da eseguire e i valori che cambiano durante il calcolo.

Vogliamo calcolare $5 \cdot 3$ usando soltanto somme. Possiamo sommare 5 tre volte: il risultato è sempre 15, ma il procedimento dipende dalle operazioni che l'esecutore sa compiere.

Distinguiamo quindi due domande: quale risultato vogliamo? Quali passi possono produrlo?

1.1 Dal problema alla specifica

«Trova il numero di telefono di una persona» sembra una richiesta chiara. Ma che cosa accade se due persone hanno lo stesso nome? E se il nome non compare? Per programmare dobbiamo sciogliere queste ambiguità prima di scegliere i passi del calcolo.

Una **specifica** descrive quali dati sono ammessi all'inizio e quale relazione deve esserci fra questi dati e il risultato. Per il prodotto: dati due interi non negativi a, b, ottenere $a\,b$. Abbiamo esteso l'esempio positivo delle lezioni al caso zero, che potremo controllare esplicitamente. La specifica non impone ancora somme, moltiplicazioni o altri strumenti.

Per precisare un problema possiamo chiederci quali dati servono, quali parti possiamo affrontare separatamente e se abbiamo già risolto qualcosa di simile. Sono tre aspetti del **pensiero computazionale**: *astrazione*, *scomposizione* e riconoscimento di *schemi*.

Il passo successivo è scrivere una procedura così precisa da poter essere eseguita automaticamente. L'informatica studia come progettare e analizzare queste procedure, chiamate algoritmi, e quali problemi possiamo risolvere con esse.

Concetto chiave · Algoritmo e programma
Un algoritmo descrive un procedimento mediante regole finite, non ambigue ed eseguibili da un esecutore adatto. Un programma lo esprime in un linguaggio di programmazione. Per un problema che richiede un risultato, chiediamo anche che l'esecuzione termini su tutti i dati ammessi e produca quel risultato.

Una descrizione finita può contenere una ripetizione: «finché rimangono somme, eseguine una» è una sola regola, anche se viene usata molte volte. La finitezza della descrizione e la terminazione dell'esecuzione sono quindi due questioni distinte. «Aggiungi sale quanto basta» lascia invece indeterminato il controllo da eseguire: per un esecutore automatico occorre una condizione precisa.

Una volta precisata la *specifica*, progettiamo l'algoritmo. Poi lo traduciamo in un linguaggio, cioè lo *codifichiamo*, e lo eseguiamo.

Se qualcosa non funziona, cerchiamo dove nasce l'errore: nella richiesta, nel procedimento, nella traduzione o nell'esecuzione. Questa ricerca si chiama **debugging**; un debugger aiuta a osservare i singoli passi.

Un test riuscito conferma il risultato sul caso provato. Per garantire tutti i casi ammessi servirà invece una dimostrazione.

1.2 Stato e assegnamento: che cosa cambia

Nel prodotto per somme ci servono tre informazioni: il valore da aggiungere, quante aggiunte restano e quanto abbiamo già accumulato. Chiamiamole A, B, C. Uno stato raccoglie i loro valori in un certo momento:

$$\sigma = \{A \mapsto 5, B \mapsto 3, C \mapsto 0\}.$$

La scrittura $B \mapsto 3$ associa il nome B al valore 3. Il nome permette di ritrovare l'informazione; il valore può cambiare.

Ogni nome presente deve avere un solo valore. Alcuni nomi possono invece mancare: diciamo perciò che lo stato è una **funzione parziale** dai nomi ai valori.

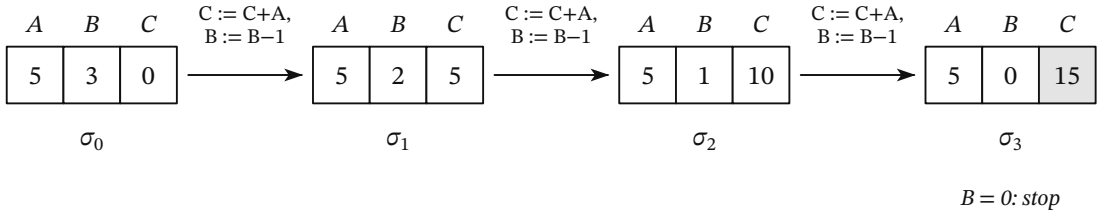
Lo stato $\{x \mapsto 3, y \mapsto 3\}$ è ammesso: nomi diversi possono avere lo stesso valore. $\{x \mapsto 3, x \mapsto 4\}$ non lo è: quale valore dovremmo leggere per x?

L'**assegnamento** $C := C + A$ legge i valori correnti di C e A, ne calcola la somma e sostituisce il valore di C. Nello stato precedente ottiene $C = 5$. Non è un'equazione da risolvere: i due usi di C riguardano due momenti diversi. Il valore di A resta 5 e quello di B resta 3.

Possiamo dunque scrivere il procedimento:

```
C := 0;
while B > 0 do
  C := C + A;
  B := B - 1
endw
```

La guardia $B > 0$ viene controllata prima di ogni giro. Se è vera, si eseguono nell'ordine le due istruzioni; poi si torna al controllo. Se è falsa, si esce senza eseguire il corpo. Con $B = 0$ all'inizio il risultato resta 0: il caso zero è già compreso.



Stati ai controlli della guardia per $A = 5, B = 3$. Tra due colonne si eseguono prima l'aggiunta e poi il decremento.

Perché diminuire B? Per ricordare quante aggiunte mancano. Perché iniziare C da zero? Prima della prima aggiunta non abbiamo ancora accumulato nulla. Dopo il primo giro abbiamo fatto una somma e ne restano due; dopo il secondo abbiamo 10 e ne resta una; dopo il terzo abbiamo 15 e non ne resta nessuna.

1.2.1 Ricordare il prodotto iniziale

Per controllare il risultato dobbiamo ricordare i valori di partenza. Chiamiamo a il valore iniziale di A e b quello di B. Sono **valori di specifica**: restano fissati mentre il programma cambia le sue variabili.

Con $a = 5, b = 3$, dopo il primo giro abbiamo prodotto 5 e restano due aggiunte da 5. La parte già calcolata e quella da calcolare danno ancora il prodotto iniziale:

$$5 + 5 \cdot 2 = 15.$$

In generale, ai controlli della guardia vogliamo conservare

$$C + A\,B = a\,b.$$

All'inizio è vera, perché $C = 0, A = a, B = b$. In un giro aggiungiamo A a C e togliamo 1 a B. L'aumento della parte calcolata compensa la diminuzione della parte rimasta:

$$(C + A) + A(B - 1) = C + A\,B.$$

Quando $B = 0$, l'uguaglianza diventa $C = a\,b$: è il risultato cercato. Quel momento viene raggiunto perché B parte da un naturale e scende di 1 finché arriva a zero.

Una proprietà che si conserva in questo modo si chiama **invariante**. La ritroveremo nei capitoli 6 e 7, dove impareremo a dimostrarla e a usarla nelle prove dei cicli.

Non basta scrivere alla fine «$C = A\,B$»: nello stato finale $B = 0$, quindi quel prodotto non ricorda più l'ingresso. La richiesta corretta è $C = a\,b$. Se vogliamo preservare anche gli ingressi, dobbiamo dichiararlo nella specifica e usare un contatore ausiliario.

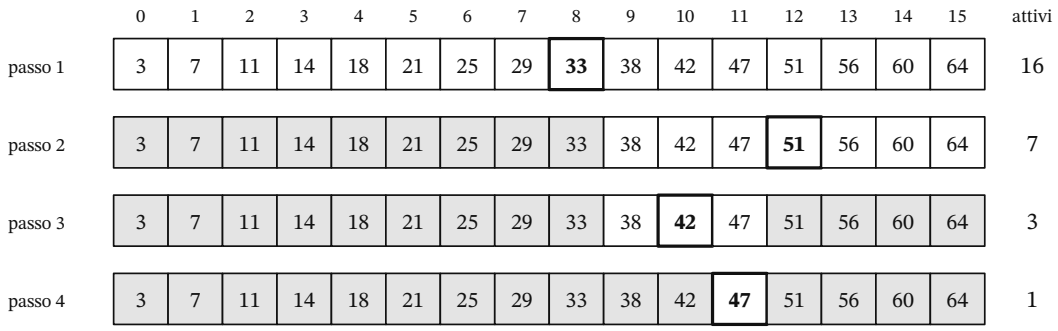
1.3 Algoritmi diversi, costi diversi

Il prodotto per somme impiega b giri. Possiamo sfruttare una regolarità per ridurli. Se il primo fattore è pari, dimezzarlo e raddoppiare il secondo conserva il prodotto. Se è dispari e maggiore di 1, togliamo un'unità e mettiamo da parte una copia del secondo fattore. Con $13 \cdot 11$ otteniamo:

$$13 \cdot 11 = 12 \cdot 11 + 11 = 6 \cdot 22 + 11 = 3 \cdot 44 + 11.$$

Ora $3 \cdot 44 = 2 \cdot 44 + 44 = 1 \cdot 88 + 44$, quindi il risultato è $88 + 44 + 11 = 143$. È l'idea della moltiplicazione egizia richiamata nella lezione introduttiva: un procedimento antico mette in luce come usare proprietà dei numeri per progettare un algoritmo.

Un secondo esempio è la ricerca in un elenco ordinato. Confrontiamo il nome cercato con quello centrale. Se il cercato viene prima, tutti i nomi successivi possono essere esclusi; se viene dopo, escludiamo tutti i precedenti. L'ordine giustifica lo scarto. Senza ordinamento, eliminare una metà non sarebbe lecito.



Ricerca di 47: i valori confrontati sono 33, 51, 42, 47. La zona attiva si restringe perché i valori sono ordinati.

Qui contiamo un confronto a tre esiti con l'elemento centrale come un passo. Il numero dei candidati si riduce circa della metà, mentre in una ricerca sequenziale si esaminano uno dopo l'altro. Dire che un algoritmo è corretto non dice ancora quanto costa eseguirlo: **correttezza** ed **efficienza** sono proprietà diverse.

1.4 Modelli di calcolo e limiti

Possiamo rendere precisa l'idea di esecutore senza costruire un calcolatore fisico. Una **macchina di Turing** usa un nastro di simboli, una testina e regole finite; può leggere, scrivere e cambiare posizione e stato di controllo.

La **macchina universale** riceve la descrizione di un'altra macchina e il suo ingresso e ne simula il comportamento: uno stesso esecutore può svolgere programmi diversi.

1.4.1 Modi diversi di descrivere il calcolo

Il **λ-calcolo** di Church descrive il calcolo attraverso funzioni e applicazioni. Ha una forma diversa dalla macchina di Turing, ma i due modelli calcolano le stesse funzioni: la loro equivalenza è un teorema.

La **tesi di Church–Turing** afferma che questi modelli descrivono tutti i procedimenti che intendiamo intuitivamente come calcoli eseguibili. La chiamiamo tesi perché la nozione intuitiva non ha una definizione matematica indipendente.

L'architettura di **von Neumann** conserva dati e istruzioni in memoria. Quando discutiamo la sua capacità astratta di calcolo assumiamo una memoria estendibile; un calcolatore reale ha invece risorse finite.

Il corso cita anche sistemi di Post e funzioni ricorsive come altri modelli di calcolo. I modelli quantistici e biologici sono qui soltanto richiamati: il volume non ne confronta l'efficienza.

1.4.2 Una richiesta che nessun algoritmo risolve in generale

Non ogni richiesta precisa ammette un algoritmo generale. Il **problema dell'arresto** chiede di decidere, per ogni programma e ogni ingresso, se l'esecuzione terminerà. È indecidibile: non esiste un algoritmo che dia sempre la risposta corretta e termini su tutti questi casi. Ciò non impedisce di dimostrare la terminazione di specifici programmi, come faremo nei capitoli 7 e 8.

1.4.3 Come si organizza un programma

Possiamo organizzare il calcolo in modi diversi, chiamati **paradigmi di programmazione**. La tabella raccoglie quelli citati nel corso.

Paradigma	Idea principale
Imperativo, come C	Comandi che cambiano lo stato
Funzionale	Funzioni e applicazioni
A oggetti	Oggetti che raccolgono dati e operazioni
Logico, come Prolog	Relazioni e inferenze
Concorrente	Processi che interagiscono

Ci concentreremo sul paradigma imperativo. Prima di studiarne gli altri comandi, dobbiamo imparare a esprimere con precisione le proprietà degli stati: è il compito della logica.

VERIFICA 1

1. Nell'algoritmo del prodotto, parti da $A = 4, B = 2$. Scrivi gli stati ai controlli della guardia, spiega il ruolo di C e indica perché $C = A\,B$ non è la richiesta finale corretta.

2. Vuoi restituire il maggiore fra due interi, conservando gli ingressi. Scrivi una specifica che distingua ingressi, risultato e vincoli. Spiega perché la richiesta non stabilisce ancora l'algoritmo.

Calcolo proposizionale: verità e conseguenza

In breve
Quando una conclusione segue davvero dalle premesse? Sappiamo già ragionare con frasi come «se piove, porto l'ombrello». Impareremo a controllare questi ragionamenti e a scrivere con precisione le condizioni che useremo nei programmi.

«Se Corrado va al cinema, ci va Antonio; se Antonio va al cinema, ci va Bruno.» Possiamo concludere che, se ci va Corrado, ci va Bruno? Sì: quando Corrado va, la prima premessa porta ad Antonio e la seconda a Bruno.

Non possiamo invece concludere che, se va Bruno, vada Corrado. Bruno potrebbe andare da solo. Il calcolo proposizionale rende preciso questo controllo: una conclusione segue se non possiamo avere premesse vere e conclusione falsa.

2.1 Formule, connettivi e interpretazioni

Una **proposizione** è una frase dichiarativa alla quale attribuiamo vero o falso. Una domanda e un comando non sono proposizioni. «$x + 1 = 2$» diventerà vera o falsa quando avremo fissato x; torneremo su questa dipendenza con il primo ordine.

Per ora rappresentiamo le proposizioni semplici con lettere $p, q, r, \ldots$. Da queste costruiamo formule: $\neg P$ significa «non P», $P \wedge Q$ «P e Q», $P \vee Q$ «almeno una fra P e Q». La disgiunzione è inclusiva: ammette che entrambe siano vere. $P \Rightarrow Q$ significa «se P, allora Q»; $P \equiv Q$ «P se e solo se Q».

Il significato di «se» merita una sosta. «Se piove, porto l'ombrello» esclude il solo caso in cui piove e non porto l'ombrello. Quando non piove, l'affermazione non impone nulla sull'ombrello. Perciò $P \Rightarrow Q$ è falsa soltanto con P vera e Q falsa. L'equivalenza, invece, è vera quando i due valori coincidono.

Indichiamo i valori con t e f. Questa tabella definisce le operazioni booleane:

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \equiv Q$
t	t	f	t	t	t	t
t	f	f	f	t	f	f
f	t	t	f	t	t	f
f	f	t	f	f	t	t

Quali scritture sono formule? Partiamo dalle lettere e dalle costanti T, F. Se abbiamo già due formule, possiamo collegarle con $\wedge$, $\vee$, $\Rightarrow$ oppure $\equiv$; se ne abbiamo una, possiamo negarla.

Ripetiamo le stesse regole sui risultati ottenuti: questa costruzione si dice **ricorsiva**. Le regole formano la **sintassi** del linguaggio e si possono raccogliere in una grammatica libera dal contesto. $p \wedge \vee$ non è una formula, perché dopo $\wedge$ manca una seconda formula.

Le costanti T e F indicano rispettivamente la formula sempre vera e quella sempre falsa. A differenza delle lettere, non hanno un valore da scegliere.

Le parentesi descrivono la costruzione. Nel corso $\neg$ lega più di $\wedge, \vee$, che legano più di $\Rightarrow$, che lega più di $\equiv$. Fra $\wedge$ e $\vee$ non scegliamo una precedenza: scriviamo le parentesi. Anche due implicazioni consecutive richiedono parentesi. Con P, Q false, $(P \Rightarrow \neg P) \Rightarrow Q$ è falsa, mentre $P \Rightarrow (\neg P \Rightarrow Q)$ è vera. Le parentesi hanno cambiato la domanda logica.

Un'**interpretazione** I assegna a ogni lettera un valore booleano. La semantica valuta poi la formula seguendone la costruzione. Con $I(p) = \mathrm{t}$, $I(q) = I(r) = \mathrm{f}$, per valutare $(p \wedge q) \vee \neg r$ calcoliamo prima $p \wedge q$, che è falsa, e $\neg r$, che è vera; la loro disgiunzione è vera. Scriviamo $[\![P]\!]_I$ per il valore di P in I.

Per una **tavola di verità** ripetiamo la valutazione per tutte le scelte dei valori. Con n lettere distinte ci sono 2^n scelte, perché ogni lettera ha due possibilità indipendenti.

2.2 Modelli, tautologie e controesempi

Un'interpretazione che rende vera P è un suo **modello**. Scriviamo $I \vDash P$ per dire che I è un modello di P.

Una formula è **soddisfacibile** se ha almeno un modello. Per esempio, $p \Rightarrow q$ è vera quando p, q sono vere: questo basta a mostrarla soddisfacibile.

Una **tautologia** è vera in ogni interpretazione. $p \vee \neg p$ lo è: qualunque valore abbia p, una delle due parti è vera. $p \Rightarrow q$ invece non lo è, perché fallisce con p vera e q falsa.

Una **contraddizione** non ha modelli. $p \wedge \neg p$ lo è: non possiamo rendere vere insieme una proposizione e la sua negazione.

CONCETTO CHIAVE · Conseguenza logica
$\Gamma \vDash Q$ significa che ogni interpretazione che rende vere tutte le premesse di Γ rende vera anche Q. Per confutare la conseguenza basta un **controesempio**: premesse tutte vere e conclusione falsa.

Riprendiamo gli amici: $c \Rightarrow a$ e $a \Rightarrow b$. La conclusione $c \Rightarrow b$ segue: se fosse falsa avremmo c vera e b falsa. La prima premessa imporrebbe a vera, e la seconda sarebbe allora falsa. Non esiste quindi un controesempio. Per $b \Rightarrow c$, invece, scegliamo b vera, a, c false: entrambe le premesse sono vere, la conclusione no.

Non concludiamo neppure che nessuno va al cinema: l'interpretazione con tutti e tre veri soddisfa le premesse. Una condizione necessaria non è automaticamente sufficiente. «Antonio va *solo se* va Bruno» si scrive $a \Rightarrow b$: Antonio che va obbliga Bruno, non viceversa.

Con premesse finite $P_1, \ldots, P_n$, la conseguenza equivale alla tautologicità di $(P_1 \wedge \cdots \wedge P_n) \Rightarrow Q$. Infatti la formula è falsa esattamente nei controesempi. Questo permette di controllare le inferenze con una tavola di verità, ma il numero delle righe cresce rapidamente. Le leggi ci danno un altro modo di ragionare.

2.3 Equivalenze e dimostrazioni motivate

Una **legge** è una tautologia che usiamo per giustificare passaggi. Se $P \equiv Q$ è una legge, possiamo sostituire una sottoformula P con Q dentro una formula più grande: i valori restano uguali in ogni interpretazione. È il principio di sostituzione per l'equivalenza.

Per esempio $P \Rightarrow Q$ si può sostituire con $\neg P \vee Q$, perché entrambe sono false nel solo caso P vera, Q falsa.

2.3.1 Leggi per trasformare una formula

Scambiare l'ordine di due termini non cambia una congiunzione o una disgiunzione: è la **commutatività**. Se ripetiamo lo stesso connettivo, possiamo anche spostare le parentesi: è l'**associatività**.

Ripetere la stessa condizione non aggiunge richieste. Questa proprietà si chiama **idempotenza**:

$$P \wedge P \equiv P, \qquad P \vee P \equiv P.$$

Le costanti T, F permettono altre semplificazioni. Aggiungere una condizione già vera a una congiunzione non cambia nulla; aggiungere una possibilità sempre falsa a una disgiunzione neppure:

$$P \wedge T \equiv P, \qquad P \vee F \equiv P.$$

Sono le leggi di **unità**. Se invece una congiunzione contiene una parte falsa, fallisce tutta; se una disgiunzione contiene una parte vera, è vera tutta:

$$P \wedge F \equiv F, \qquad P \vee T \equiv T.$$

Qui F e T sono detti **elementi assorbenti** per le rispettive operazioni.

La **distributività** consente di separare i due casi di $Q \vee R$:

$$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R).$$

Vale anche la legge ottenuta scambiando $\wedge$ e $\vee$. Useremo queste leggi quando ci aiutano ad avvicinare parti che possiamo semplificare.

La doppia negazione dà $\neg\neg P \equiv P$. Le leggi di De Morgan dicono

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q, \qquad \neg(P \vee Q) \equiv \neg P \wedge \neg Q.$$

Per capire la prima, chiediamoci quando fallisce «entrambi»: basta che fallisca almeno uno. La seconda dice che per fallire «almeno uno» devono fallire entrambi. Inoltre $\neg(P \Rightarrow Q) \equiv P \wedge \neg Q$, ed $P \equiv Q$ equivale sia a $(P \Rightarrow Q) \wedge (Q \Rightarrow P)$ sia a $(P \wedge Q) \vee (\neg P \wedge \neg Q)$.

Vediamo perché può essere utile distribuire, anziché farlo senza una meta. Vogliamo semplificare $P \vee (\neg P \wedge Q)$. Qui P compare anche negata: distribuendo portiamo le due occorrenze vicine:

$$P \vee (\neg P \wedge Q) \equiv (P \vee \neg P) \wedge (P \vee Q) \equiv T \wedge (P \vee Q) \equiv P \vee Q.$$

Il primo passo usa la distributività; il secondo usa $P \vee \neg P \equiv T$, il *terzo escluso*; l'ultimo usa l'unità. Abbiamo ottenuto una **legge di complemento**. L'altra è $P \wedge (\neg P \vee Q) \equiv P \wedge Q$, proposta nella verifica finale.

Le leggi di **assorbimento** danno invece

$$P \wedge (P \vee Q) \equiv P, \qquad P \vee (P \wedge Q) \equiv P.$$

Nella prima, chiedere P rende già vera $P \vee Q$. Nella seconda, il caso $P \wedge Q$ era già compreso nel caso P. Non occorre memorizzare tutto insieme: cerchiamo ogni volta la legge che rende più semplice la formula.

2.3.2 Dalle premesse alla conclusione

Per dimostrare un'implicazione possiamo assumere la premessa e ricavare la conclusione. Da $P \Rightarrow Q$ e P segue Q: è il **modus ponens**, perché la sola possibilità di Q falsa renderebbe falsa una premessa.

Da $P \Rightarrow Q$ e $\neg Q$ segue $\neg P$ (**modus tollens**): se P fosse vera, costringerebbe Q a esserlo. Qui abbiamo usato un ragionamento per assurdo, non una nuova operazione misteriosa.

La **controposizione** esprime proprio $(P \Rightarrow Q) \equiv (\neg Q \Rightarrow \neg P)$. Non scambia semplicemente i due lati: li nega entrambi. La prova per assurdo di $P \Rightarrow Q$ cerca una contraddizione assumendo $P \wedge \neg Q$, l'unico caso che potrebbe falsificare l'implicazione. Una prova per casi sceglie una proposizione R e dimostra la stessa conclusione sia con R sia con $\neg R$; questi due casi esauriscono le possibilità.

2.4 Implicazione e polarità delle occorrenze

Sappiamo che $x > 10 \Rightarrow x > 0$. La condizione «maggiore di 10» è più forte: ammette meno numeri. Se la sostituiamo con «maggiore di 0» nella richiesta $x > 10 \wedge R$, otteniamo una richiesta più debole:

$$(x > 10 \wedge R) \Rightarrow (x > 0 \wedge R).$$

Sotto una negazione accade il contrario. «Non maggiore di 0» implica «non maggiore di 10», mentre il viceversa fallisce per $x = 5$:

$$\neg(x > 0) \Rightarrow \neg(x > 10).$$

La negazione ha invertito il verso. Anche stare a sinistra di un'implicazione lo inverte. La richiesta $A \Rightarrow R$ diventa più impegnativa se sostituiamo A con una condizione più debole: dovremo garantire R in più casi.

2.4.1 Seguire un'occorrenza nella formula

Prendiamo una sola formula e seguiamo tutti i passaggi:

$$\Phi = \neg(P \Rightarrow Q) \wedge R.$$

Possiamo leggerne la costruzione come un albero. La congiunzione è il punto di partenza; a sinistra troviamo una negazione, poi un'implicazione. A destra troviamo direttamente R.

Occorrenza	Percorso dalla congiunzione	Inversioni
P	$\wedge, \neg,$ antecedente di $\Rightarrow$	2
Q	$\wedge, \neg,$ conseguente di $\Rightarrow$	1
R	$\wedge$	0

Per arrivare a P attraversiamo prima $\neg$ e poi il lato sinistro di $\Rightarrow$. Sono due inversioni: il verso iniziale si ristabilisce. Diciamo che questa occorrenza è **positiva**.

Per arrivare a Q attraversiamo la stessa negazione, ma poi il lato destro di $\Rightarrow$, che non inverte. Rimane una sola inversione: l'occorrenza è **negativa**. Per R non incontriamo inversioni: è positiva.

Un controllo è riscrivere Φ come $(P \wedge \neg Q) \wedge R$. Ora i tre ruoli si vedono direttamente.

> **CONCETTO CHIAVE · Polarità di un'occorrenza**
> Partiamo dal segno positivo e seguiamo il percorso fino all'occorrenza scelta. Ogni negazione e ogni passaggio nell'antecedente di $\Rightarrow$ cambia il segno. $\wedge$, $\vee$ e il conseguente lo lasciano invariato. Prima eliminiamo gli eventuali $\equiv$.

Conta il posto occupato da ciascuna occorrenza, non il nome della lettera: due P nella stessa formula possono avere segni diversi.

2.4.2 Usare il segno per sostituire

Supponiamo valida $A \Rightarrow B$. In una posizione positiva, sostituire A con B dà un'implicazione dalla formula iniziale alla nuova. In una posizione negativa otteniamo l'implicazione dalla nuova alla iniziale.

Per esempio, $Q \wedge R \Rightarrow Q$. Nel conseguente di $P \Rightarrow (Q \wedge R)$ non abbiamo inversioni, quindi

$$(P \Rightarrow (Q \wedge R)) \Rightarrow (P \Rightarrow Q).$$

Invece $P \Rightarrow P \vee Q$. La sostituzione nell'antecedente di $P \Rightarrow R$ incontra una inversione e dà il verso opposto:

$$((P \vee Q) \Rightarrow R) \Rightarrow (P \Rightarrow R).$$

La formula a sinistra garantisce R sia quando vale P sia quando vale Q. Perciò garantisce anche il solo caso P.

2.4.3 Tenere presenti le ipotesi

Se abbiamo assunto P, possiamo sostituirla con T e scrivere $P \wedge Q \equiv T \wedge Q \equiv Q$. Il risultato dipende però dall'ipotesi. Abbiamo dimostrato

$$P \Rightarrow ((P \wedge Q) \equiv Q),$$

non $(P \wedge Q) \equiv Q$ senza condizioni. Con P falsa e Q vera quest'ultima sarebbe falsa.

2.4.4 Per proseguire: forme normali, risoluzione e NAND

Un **letterale** è una lettera o la sua negazione, come p e $\neg q$. Una **forma normale congiuntiva** (FNC) è una congiunzione di disgiunzioni di letterali:

$$(p \vee \neg q) \wedge r.$$

Una **forma normale disgiuntiva** (FND) scambia i ruoli: è una disgiunzione di congiunzioni di letterali, come $(p \wedge q) \vee \neg r$.

Per ottenere queste forme eliminiamo $\Rightarrow, \equiv$, spingiamo le negazioni fino alle lettere con De Morgan e distribuiamo. Il procedimento può allungare molto le formule; la forma ottenuta non è unica.

2.4.5 Risolvere due clausole

La **risoluzione** usa due clausole $P \vee Q$ e $\neg P \vee R$ per ricavare $Q \vee R$. Perché? Se P è falsa, la prima clausola impone Q; se è vera, la seconda impone R. È una prova per casi.

Dalle tre clausole $P \vee Q$, $\neg P \vee R$, $\neg Q \vee R$ ricaviamo prima $Q \vee R$, poi, risolvendo su Q, R. La risoluzione è uno strumento dei metodi automatici e della programmazione logica.

2.4.6 Quanti connettivi sono necessari?

Un insieme di connettivi è **funzionalmente completo** se permette di esprimere ogni funzione booleana, cioè ogni scelta dei risultati in una tavola di verità.

L'insieme $\{\neg, \wedge\}$ è completo. De Morgan permette di esprimere $\vee$; con una FND possiamo poi raccogliere precisamente le righe della tavola in cui vogliamo il risultato vero. Per la funzione sempre falsa basta $P \wedge \neg P$.

Anche $\{\neg, \vee\}$ è completo, per De Morgan. Lo è $\{\neg, \Rightarrow\}$, perché possiamo costruire $P \vee Q$ come $\neg P \Rightarrow Q$.

Basta persino il solo **NAND**, definito da $P \uparrow Q \equiv \neg(P \wedge Q)$. Prima ricostruiamo la negazione, poi la congiunzione:

$$P \uparrow P \equiv \neg P, \qquad (P \uparrow Q) \uparrow (P \uparrow Q) \equiv P \wedge Q.$$

Abbiamo così ritrovato l'insieme completo $\{\neg, \wedge\}$. Usare più connettivi rimane utile per scrivere formule leggibili.

Verifica 2

1. Con p: «piove» ed e: «esco», traduci «esco solo se non piove». La formula ottenuta implica «se non piove, esco»? Cerca un caso che distingua le due richieste.

2. Semplifica $P \wedge (\neg P \vee Q)$ mostrando le leggi usate. Poi stabilisci il segno delle occorrenze di P, Q, R in $\neg(P \Rightarrow Q) \wedge R$.

Logica del primo ordine: individui e formule

In breve
Come diciamo che una proprietà vale per tutti i numeri, oppure soltanto per alcuni? Le lettere proposizionali da sole non mostrano gli oggetti di cui parliamo. Introduciamo variabili, funzioni e quantificatori per esprimere queste richieste.

«Tutti i numeri pari hanno successore dispari.» Potremmo introdurre una lettera per il caso di 0, una per il caso di 2, e così via, ma non avremmo scritto la regola generale.

Ci serve distinguere gli oggetti e le operazioni: il numero, la proprietà «pari», l'operazione «successore». La **logica del primo ordine** aggiunge questi ingredienti e permette di dire «per ogni» ed «esiste».

3.1 Termini, predicati e quantificatori

Scegliamo prima gli oggetti di cui parlare, il **dominio**. Per l'esempio usiamo $\mathbb{N} = \{0, 1, 2, \dots\}$. Introduciamo la costante 1, la funzione somma, i predicati pari e dispari, e una variabile x. La scrittura

$$\forall x.(\text{pari}(x) \Rightarrow \text{dispari}(x+1))$$

dice che, qualunque numero sia scelto per x, se è pari il suo successore è dispari. Il simbolo $\forall$ è il **quantificatore universale**. Per dire che almeno un numero è pari usiamo l'**esistenziale** $\exists x.\text{pari}(x)$.

Per scrivere queste formule servono nomi per numeri fissati (**costanti**), posti per numeri ancora da scegliere (**variabili**), operazioni (**funzioni**) e proprietà o relazioni (**predicati**). Sono gli ingredienti del linguaggio del primo ordine, insieme a connettivi e quantificatori.

Ogni funzione e ogni predicato richiede un certo numero di argomenti, chiamato **arietà**. Il successore ne richiede uno, la somma due, il predicato «minore» due. Un predicato senza argomenti, di arietà zero, si comporta come una lettera proposizionale.

La scrittura $s(s(x))$ indica il successore del successore di x. Indica un numero: non è ancora una frase vera o falsa. Una scrittura che indica un oggetto si chiama **termine**.

Ogni costante e ogni variabile è un termine. Possiamo poi applicare una funzione ai termini già costruiti: se f richiede n argomenti, da $t_1, \dots, t_n$ otteniamo il termine $f(t_1, \dots, t_n)$.

Nell'esempio applichiamo prima s a x, ottenendo $s(x)$, poi ancora s al risultato. La stessa regola si usa due volte.

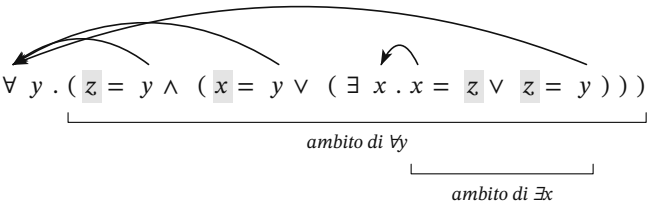
Le **formule atomiche** applicano un predicato a termini: $\text{pari}(s(s(x)))$ oppure $x < y$. Da queste, da T, F, e dai connettivi costruiamo formule come prima. La nuova regola consente, data una formula P, di costruire $\forall x.P$ ed $\exists x.P$. Quantificatori consecutivi uguali si possono abbreviare, per esempio $\forall x, y.P$.

Concetto chiave · Termine e formula
Un termine indica un individuo; una formula esprime una condizione vera o falsa. Le funzioni prendono individui e restituiscono individui; i predicati prendono individui e restituiscono valori booleani. $x + 1$ è un termine, $\text{pari}(x+1)$ una formula. $\text{pari}(x) + 1$ non rispetta questi ruoli.

3.2 Portata, variabili libere e sostituzione

In $\forall x.P$ la formula P è la **portata** del quantificatore. Un'occorrenza di x è **legata** quando si trova nella portata di un quantificatore su x; altrimenti è **libera**. Se vi sono quantificatori annidati con lo stesso nome, conta quello più interno che comprende l'occorrenza.

In $(\forall x.p(x)) \wedge q(x, y)$ la prima x è legata; la seconda x e y sono libere. Dunque una stessa lettera può avere occorrenze con ruoli diversi. Una formula è **chiusa** se non ha variabili libere, **aperta** altrimenti. La formula $x + 1 = 2$ dipende dal valore esterno di x; $\exists x.x + 1 = 2$ afferma invece l'esistenza di un valore adatto.



Le frecce identificano il quantificatore che lega ogni occorrenza. Nella formula illustrata tutte le y sono legate, tutte le z libere; la x esterna a $\exists x$ rimane libera.

La notazione $P[t/x]$ significa: sostituisci il termine t alle occorrenze *libere* di x in P. Non cambiamo le occorrenze già governate da un quantificatore interno su x. Inoltre nessuna variabile libera del termine inserito deve diventare legata per effetto della sostituzione: sarebbe una **cattura**.

Vediamo un caso che spiega la precauzione. In $P = \exists y.x < y$ vogliamo sostituire y a x. Scrivere $\exists y.y < y$ sarebbe sbagliato: la y inserita, che doveva restare libera, è stata catturata. Prima rinominiamo il segnaposto legato: $\exists z.x < z$. Ora sostituiamo e otteniamo $\exists z.y < z$. Sui naturali questa formula dice correttamente che il numero y ha un numero più grande.

Possiamo quindi cambiare il nome di una variabile legata, purché il nuovo nome non crei altri legami. Questa operazione è la **ridenominazione delle variabili legate**.

Se z non compare in P, possiamo scrivere

$$\forall x.P \equiv \forall z.P[z/x].$$

Nella portata P sostituiamo soltanto le occorrenze libere di x, cioè quelle governate dall'universale esterno. Il nome è un segnaposto: conta quali occorrenze dipendono dal quantificatore.

3.3 Interpretazioni: i simboli ricevono un significato

La formula $\forall x.p(x)$ non dice ancora quali oggetti consideriamo né quale proprietà esprima p. Dobbiamo scegliere un dominio non vuoto D e dare un significato ai simboli. Queste scelte formano un'**interpretazione**, che scriviamo $I = (D, \alpha)$.

Ogni costante indica un elemento di D. Una funzione con n argomenti prende n elementi del dominio e ne restituisce uno. Deve essere definita su tutte le scelte: è una funzione *totale* $D^n \to D$.

Un predicato con n argomenti prende gli stessi dati, ma restituisce vero o falso: è una funzione $D^n \to \text{Bool}$. Qui D^n raccoglie le liste ordinate di n elementi di D. Per l'uguaglianza useremo il significato usuale: due termini sono uguali se indicano lo stesso individuo.

Il dominio conta anche per gli elementi senza nome. Se $D = \{5, 10, 15\}$ e p significa «essere multiplo di 5», $\forall x.p(x)$ è vera. Se ampliamo il dominio a $\mathbb{N}$ e manteniamo lo stesso significato, la formula è falsa: 7 è un controesempio. Avere costanti che nominano soltanto 5, 10 e 15 non elimina gli altri numeri dal dominio dei quantificatori.

Costanti diverse possono indicare lo stesso individuo, se non imponiamo che siano distinti. Simboli dall'aspetto familiare, come $+$, non hanno automaticamente l'interpretazione aritmetica: occorre dichiararla. Nel seguito, quando lavoriamo sui numeri e sui programmi, useremo esplicitamente quella usuale.

3.4 Formalizzare: tutti, alcuni, esattamente uno

Prendiamo come dominio le persone e come predicati $S(x)$: «x segue FPL», $C(x)$: «x va al cinema».

«Tutti gli studenti di FPL vanno al cinema» diventa $\forall x.(S(x) \Rightarrow C(x))$. Una persona che non segue FPL non è un controesempio: l'enunciato non la riguarda.

«Qualche studente di FPL va al cinema» diventa invece $\exists x.(S(x) \wedge C(x))$: il testimone deve soddisfare entrambe le condizioni.

Questo spiega le abbreviazioni della **quantificazione ristretta**:

$$(\forall x\colon S(x).C(x)) \equiv (\forall x.(S(x) \Rightarrow C(x))),$$

$$(\exists x\colon S(x).C(x)) \equiv (\exists x.(S(x) \wedge C(x))).$$

Usare un'implicazione nell'esistenziale darebbe un significato diverso: basterebbe una persona che non segue FPL, anche se nessuno studente andasse al cinema.

Usare una congiunzione nell'universale imporrebbe invece che tutte le persone seguano FPL.

3.4.1 Dire che l'eccezione è una sola

«Tutti tranne uno» richiede sia un'eccezione sia la sua unicità. Costruiamo prima la proprietà dell'eccezione, $E(x) = S(x) \wedge \neg C(x)$. Dire «esattamente uno studente di FPL non va al cinema» significa

$$\exists x.(E(x) \wedge \forall y.(E(y) \Rightarrow y = x)).$$

La prima parte dà un'eccezione; la seconda impone che ogni eccezione coincida con essa. Equivalentemente possiamo dire che tutti gli altri studenti vanno al cinema. Senza la seconda parte avremmo detto soltanto «almeno uno».

3.4.2 Una relazione fra più individui

Il dominio e il vocabolario si scelgono in funzione della domanda. Per «due persone sono parenti se hanno un antenato comune» servono predicati binari $A(z, x)$: «z è antenato di x» e $R(x, y)$: «sono parenti». L'antenato senza nome diventa una variabile:

$$\forall x, y.((\exists z.(A(z, x) \wedge A(z, y))) \Rightarrow R(x, y)).$$

«Se» dà questa direzione; «se e solo se» ne imporrebbe anche un'altra. La formalizzazione richiede dunque interpretare il senso della frase, poi controllare che la formula dica proprio quello.

VERIFICA 3

1. Nella formula $(\forall x.p(x, y)) \wedge (\exists y.q(x, y))$ individua le occorrenze libere e legate. Sostituisci poi y alle occorrenze libere di x, evitando catture.

2. Sui naturali formalizza «ogni numero pari ha il successore del successore pari», usando la funzione s di successore. Formalizza poi «esattamente un elemento soddisfa p», spiegando il compito dell'uguaglianza.

Semantica e dimostrazioni con i quantificatori

In breve
Come controlliamo una formula con «per ogni» o «esiste»? Partiamo dal valore di una variabile e da piccoli domini. Capiremo poi quali passi permettono di dimostrare una proprietà generale e come usare un testimone senza attribuirgli proprietà non giustificate.

Per valutare $x+1=4$ dobbiamo sapere che numero rappresenta x. Per valutare $\exists x.x+1=4$ dobbiamo invece provare se qualche numero rende vera la condizione. La stessa idea, dare temporaneamente un valore a una variabile, ci permette di definire entrambe le valutazioni.

4.1 Assegnamenti e valore di termini e formule

Fissiamo $I=(D,\alpha)$. Un **assegnamento** ρ associa a ogni variabile un elemento di D. La scrittura $\rho[d/x]$ indica l'assegnamento uguale a ρ sulle altre variabili, ma che dà d a x.

Con $\rho(x)=2$, $\rho(y)=10$, l'assegnamento $\rho[3/x]$ dà 3 a x e ancora 10 a y.

Il valore di un termine, $\alpha_\rho(t)$, si calcola dalla sua struttura. Una variabile vale $\rho(x)$, una costante vale $\alpha(c)$; per $f(t_1,\ldots,t_n)$ calcoliamo prima i termini e poi applichiamo la funzione $\alpha(f)$ ai risultati.

Per esempio, se s è il successore sui naturali e $\rho(x)=2$, $s(s(x))$ vale 4: prima 2, poi 3, infine 4.

Per una formula atomica valutiamo i termini e applichiamo il predicato. Se p significa «essere pari», $p(s(s(x)))$ è vera con quell'assegnamento. I connettivi mantengono le tabelle del capitolo 2. Scriviamo $I_\rho(P)$ per il valore della formula. Per i quantificatori:

Concetto chiave · Semantica dei quantificatori
$I_\rho(\forall x.P)$ è vero se $I_{\rho[d/x]}(P)$ è vero per *ogni* $d\in D$. $I_\rho(\exists x.P)$ è vero se ciò accade per *almeno un* $d\in D$. Altrimenti il rispettivo valore è falso.

Su un dominio finito $\{a,b,c\}$, un universale si comporta come la congiunzione dei tre casi e un esistenziale come la loro disgiunzione. Un elemento che rende vera la portata di un esistenziale è un **testimone**. Un elemento che rende falsa quella di un universale è un controesempio.

Consideriamo $D=\{a,b\}$, con Q vera su entrambi gli elementi, e $P(x,y)$ vera soltanto quando $x=a$. Vogliamo valutare

$$\exists x.(Q(x)\wedge\forall y.P(x,y)).$$

Cerchiamo un elemento che soddisfi entrambe le richieste. Scegliamo a: $Q(a)$ è vera, e per l'universale interno controlliamo $P(a,a)$ e $P(a,b)$, entrambe vere. Dunque a è un testimone e la formula è vera. Con b la verifica fallirebbe; ciò non cancella il testimone già trovato.

$P(x,y)$	$y=a$	$y=b$
$x=a$	t	t
$x=b$	f	f

La riga di a è tutta vera: per l'universale interno serve tutta la riga, per l'esistenziale esterno ne basta una.

Una formula dipende soltanto dai valori assegnati alle sue variabili libere. Perciò una formula chiusa non dipende da ρ, e scriviamo semplicemente $I(P)$. Le nozioni di modello, soddisfacibilità e conseguenza logica si estendono come nel capitolo 2.

Una formula chiusa è **valida** se è vera in tutte le interpretazioni.

Per una formula aperta, dire che è valida significa richiedere la verità per ogni interpretazione e per ogni assegnamento alle variabili libere. Questo è il senso in cui useremo le leggi schematiche.

Essere vera sui naturali con le operazioni usuali è meno esigente: $\forall x.x+1>x$ vi è vera, ma non per qualunque interpretazione dei simboli.

4.2 Negazione, distribuzione e domini ristretti

Negare «tutti soddisfano P» significa che almeno uno non la soddisfa; negare «almeno uno soddisfa P» significa che tutti la falliscono. Sono le leggi di De Morgan:

$$\neg(\forall x.P) \equiv \exists x.\neg P, \qquad \neg(\exists x.P) \equiv \forall x.\neg P.$$

Nella quantificazione ristretta manteniamo il filtro: $\neg(\forall x\colon R.P) \equiv \exists x\colon R.\neg P$. Non neghiamo R, perché stiamo cercando proprio fra gli elementi ammessi.

Se nessun elemento soddisfa R, $\forall x\colon R.P$ è **vacuamente vera**: non ci sono controesempi ammessi. $\exists x\colon R.P$ è falsa: manca ogni testimone. In particolare, «tutti gli elementi dell'array vuoto sono positivi» non afferma che nell'array vi siano elementi.

Un universale distribuisce su $\wedge$: ogni individuo soddisfa entrambe le condizioni esattamente quando tutti soddisfano la prima e tutti la seconda. Dualmente l'esistenziale distribuisce su $\vee$:

$$(\forall x.(P \wedge Q)) \equiv ((\forall x.P) \wedge (\forall x.Q)),$$

$$(\exists x.(P \vee Q)) \equiv ((\exists x.P) \vee (\exists x.Q)).$$

4.2.1 Perché alcuni versi non si possono invertire

Un testimone che soddisfa entrambe le proprietà può essere usato separatamente per ciascuna:

$$(\exists x.(P \wedge Q)) \Rightarrow ((\exists x.P) \wedge (\exists x.Q)).$$

Il viceversa non vale. Sui naturali esistono numeri pari ed esistono numeri dispari, ma non esiste un numero che sia entrambe le cose: i due esistenziali possono avere testimoni diversi.

Per l'universale vale invece

$$((\forall x.P) \vee (\forall x.Q)) \Rightarrow \forall x.(P \vee Q).$$

Anche qui il viceversa fallisce. Ogni intero è pari o dispari; questo non significa che tutti gli interi siano pari, né che tutti siano dispari.

4.2.2 Una parte che non dipende dalla variabile

Se x non è libera in Q, il valore di Q non cambia mentre varia x. Possiamo quindi portare questa parte fuori dal quantificatore. Per esempio:

$$(\forall x.(P \wedge Q)) \equiv ((\forall x.P) \wedge Q),$$

$$(\exists x.(P \vee Q)) \equiv ((\exists x.P) \vee Q).$$

Qui usiamo il dominio non vuoto. La stessa ipotesi serve per togliere un quantificatore da una formula che non dipende affatto da x: $\forall x.Q \equiv Q$ ed $\exists x.Q \equiv Q$.

Se ammettessimo domini vuoti, $\forall x.F$ sarebbe vera e $\exists x.T$ falsa. Non potremmo quindi sostituirle con F e T.

4.2.3 L'ordine cambia la scelta del testimone

I quantificatori dello stesso tipo si scambiano: $\forall x.\forall y.P \equiv \forall y.\forall x.P$, e analogamente per due esistenziali.

Scambiare tipi diversi cambia invece la scelta che possiamo fare. Sui naturali $\forall x.\exists y.x < y$ è vera: dopo aver visto x, scegliamo $y = x + 1$.

$\exists y.\forall x.x < y$ è falsa: un solo y dovrebbe superare anche sé stesso.

Vale sempre il verso da un testimone comune a testimoni che possono dipendere da x:

$$(\exists y.\forall x.P(x, y)) \Rightarrow (\forall x.\exists y.P(x, y)).$$

4.3 Regole di prova: individui particolari e generici

Sapere che una proprietà vale per tutti permette di applicarla a un termine particolare. Da $\forall x.P$ ricaviamo $P[t/x]$: è l'**eliminazione dell'universale**.

Viceversa, se abbiamo un caso che soddisfa la proprietà, sappiamo che un caso esiste. Da $P[t/x]$ ricaviamo $\exists x.P$: è l'**introduzione dell'esistenziale**. Le sostituzioni devono sempre evitare la cattura.

Per esempio, «tutti gli uomini sono mortali» si applica a Socrate. Otteniamo «se Socrate è uomo, è mortale». Sapendo che è uomo, il modus ponens dà la conclusione.

4.3.1 Dimostrare una proprietà per tutti

Per dimostrare $\forall x.P$ non basta verificare un individuo noto. Prendiamo un nome nuovo d, senza attribuirgli proprietà particolari, e dimostriamo $P[d/x]$.

Se il ragionamento vale senza ipotesi particolari su d, possiamo concludere che vale per tutti: è la **generalizzazione**. Il nome deve essere assente dalla formula generale da dimostrare e dalle ipotesi ancora utilizzate.

Un'ipotesi è *aperta* finché il risultato dipende da essa. La *chiudiamo*, o *scarichiamo*, quando una regola permette di eliminarla dal gruppo delle premesse. L'esempio seguente mostra concretamente questa distinzione.

4.3.2 Usare un oggetto di cui sappiamo l'esistenza

Per usare $\exists x.P$ come premessa possiamo dare al testimone un nome nuovo d, assumere temporaneamente $P[d/x]$, e ricavare una conclusione che non nomini d.

Il corso chiama questo passo **skolemizzazione**; qui è la regola di uso di un testimone esistenziale. Il nome deve essere nuovo anche rispetto alla conclusione. Non stiamo affermando la legge $(\exists x.P) \Rightarrow P[d/x]$ per una costante prefissata: sarebbe falsa.

4.3.3 Una prova con un testimone temporaneo

Supponiamo di sapere che ogni oggetto con proprietà P ha anche proprietà Q, e che almeno un oggetto ha proprietà P. Vogliamo dimostrare che almeno un oggetto ha proprietà Q.

Indichiamo con Γ le due premesse, che rimangono disponibili in tutta la prova:

$$\Gamma = \{\forall x.(P(x) \Rightarrow Q(x)), \exists x.P(x)\}.$$

1. Diamo un nome al testimone. L'esistenziale non ci dice quale oggetto scegliere. Lo chiamiamo d, usando un nome nuovo, e apriamo l'ipotesi temporanea $H\colon P(d)$. Non affermiamo che una costante già nota soddisfi P.

2. Usiamo la regola generale su quell'oggetto. Dalla prima premessa, eliminando l'universale, ricaviamo $P(d) \Rightarrow Q(d)$. In questo momento le ipotesi aperte sono ancora Γ e H.

3. Ricaviamo la proprietà cercata. Con H e il modus ponens otteniamo $Q(d)$. Introducendo l'esistenziale concludiamo:

$$\Gamma, H \vdash \exists x.Q(x).$$

4. Chiudiamo l'ipotesi sul testimone. La conclusione non nomina d e nessuna premessa di Γ lo nomina. Il ragionamento funziona quindi per qualunque testimone garantito da $\exists x.P(x)$. La regola di eliminazione dell'esistenziale chiude H e lascia

$$\Gamma \vdash \exists x.Q(x).$$

Restano aperte le due premesse iniziali: la conclusione dipende da esse. Si è chiusa soltanto l'ipotesi locale $P(d)$. Non possiamo concludere $\forall x.Q(x)$: d era un testimone con una proprietà particolare, non un individuo generico.

4.3.4 Che cosa garantisce un sistema di prova

Un **sistema di dimostrazione** raccoglie le regole di inferenza che possiamo usare. Una regola consente di passare da certe premesse a una conclusione; una regola senza premesse è un assioma.

Scriviamo $\Gamma \vdash P$ quando esiste una sequenza di passi consentiti che parte dalle premesse Γ e arriva a P. Questa è una proprietà delle prove scritte, distinta da $\Gamma \vDash P$, che riguarda la verità nelle interpretazioni.

Il sistema è **corretto** se tutto ciò che possiamo derivare è conseguenza logica delle premesse. È **completo** se possiamo derivare ogni conseguenza logica.

Per il proposizionale si hanno entrambe le proprietà. Per il sistema presentato nel corso usiamo la correttezza. Esistono sistemi completi anche per il primo ordine, ma non un procedimento che decida sempre, in un numero finito di passi, la validità di una formula arbitraria.

4.3.5 Dall'ipotesi all'implicazione

Se assumiamo H e ricaviamo Q, possiamo chiudere l'ipotesi e concludere $H \Rightarrow Q$, rispettando le condizioni delle regole di quantificazione. Questo è il contenuto del **teorema di deduzione**. Le altre premesse usate restano disponibili e il risultato continua a dipendere da esse.

Anche le sostituzioni richiedono attenzione alle ipotesi. Sotto un quantificatore possiamo usare un'equivalenza, o le polarità del capitolo 2, solo se la relazione vale per tutti i valori esaminati.

Sapere $P(x)$ per il solo valore attuale di x non autorizza a sostituire $P(x)$ con T dentro $\forall x.P(x)$. Il quantificatore deve considerare anche gli altri valori.

4.4 Uguaglianza e asserzioni sugli stati

Con l'interpretazione usuale dell'uguaglianza, $t = u$ è vera quando i termini indicano lo stesso elemento. La **legge di Leibniz** dice che sostituire un individuo con uno uguale non cambia le proprietà:

$$x = y \Rightarrow (P \equiv P[y/x]).$$

La sostituzione riguarda le occorrenze libere e non deve catturare variabili. In particolare, $x = y$ insieme a P permette di concludere $P[y/x]$. Per esempio, $x = y$ ed $x > 0$ danno $y > 0$.

Le **leggi del singoletto** precisano che quantificare su un solo individuo equivale a sostituirlo. Per nomi di variabile distinti x, y, sempre evitando catture:

$$(\forall x.(x = y \Rightarrow P)) \equiv P[y/x],$$

$$(\exists x.(x = y \wedge P)) \equiv P[y/x].$$

Nella seconda formula l'unico possibile testimone è il valore di y: se soddisfa P, l'esistenziale è vero; se non la soddisfa, nessun altro valore può sostituirlo, perché violerebbe $x = y$. La prima formula controlla lo stesso unico individuo mediante un'implicazione.

Siamo pronti a parlare dei programmi. Un'asserzione come $x \geq 0 \wedge y > 0$ è una formula aperta; uno stato fornisce i valori delle variabili di programma, come un assegnamento. Scriviamo $\sigma \vDash P$ se P è vera con quei valori nell'interpretazione numerica usuale. Il prossimo capitolo userà coppie di asserzioni per distinguere ciò che chiediamo prima di un programma da ciò che vogliamo ottenere dopo.

4.4.1 Per proseguire: leggi sui domini e indebolimento

Per verificare una proprietà sull'unione di due gruppi, la verifichiamo su entrambi: $(\forall x{:}\, P \vee Q.R) \equiv ((\forall x{:}\, P.R) \wedge (\forall x{:}\, Q.R))$. Per trovare un testimone nell'unione basta trovarlo in uno dei gruppi: la legge esistenziale scambia $\wedge$ esterna con $\vee$. Anche le leggi distributive già viste e De Morgan valgono con uno stesso filtro ristretto.

L'indebolimento mantiene la direzione consueta: $(\forall x.P) \Rightarrow \forall x.(P \vee Q)$, $(\exists x.P) \Rightarrow \exists x.(P \vee Q)$, e dalle versioni con $P \wedge Q$ possiamo sempre conservare soltanto P. I quantificatori non invertono la polarità: una relazione valida per tutti gli individui può essere trasportata nelle loro portate.

Verifica 4

1. Nega $\forall x.(p(x) \Rightarrow \exists y.q(x, y))$, portando la negazione fino agli atomi. Spiega che cosa dovresti trovare per mostrare falsa la formula iniziale.

2. Su $D = \{a, b\}$, la relazione R è vera soltanto per (a, b) e (b, a). Valuta $\forall x.\exists y.R(x, y)$ ed $\exists y.\forall x.R(x, y)$. Mostra poi con un esempio perché da $P(d)$ per una costante già nota non si può sempre generalizzare a $\forall x.P(x)$.

Capitolo 5

Stati, espressioni e comandi

In breve
Che cosa accade quando eseguiamo un comando? Abbiamo già distinto il nome di una variabile dal suo valore. Ora seguiremo letture, assegnamenti, scelte e ripetizioni, precisando anche quando un'esecuzione non arriva a un risultato.

Per capire un programma dobbiamo seguire due operazioni diverse: leggere un valore e modificare un valore. Nell'espressione `x + 1` leggiamo `x` e calcoliamo un numero. Nel comando `x := x + 1` usiamo quel numero per aggiornare `x`. Il segno `:=` indica un'azione, mentre = nelle asserzioni indica un'uguaglianza.

5.1 Lo stato e la valutazione delle espressioni

Uno stato σ associa valori agli identificatori del programma. Per esempio $\sigma(x) = 10$ e $\sigma(y) = 3$. Nel linguaggio considerato i valori sono interi o booleani: Val=$\mathbb{Z} \cup$Bool. I due valori booleani sono vero e falso. Lavoriamo con interi matematici, senza un limite di memoria o fenomeni di traboccamento.

Si possono descrivere stati parziali, che non assegnano un valore a tutti i nomi possibili. Nel seguito ogni identificatore letto si assume presente nello stato e del tipo dichiarato. Questa ipotesi sarà importante quando diremo che la lettura di una variabile è sempre definita.

Le espressioni partono da costanti intere, dai valori booleani `true`, `false` e dai nomi delle variabili. Possiamo combinarle con parentesi e operatori. I nomi `true` e `false` sono riservati ai due valori booleani.

Gli operatori `+`, `-`, `*`, `div` e `mod` operano sugli interi. I confronti e gli operatori logici producono booleani.

Le precedenze usuali distinguono `8 + 5 * 3` da `(8 + 5) * 3`: il primo vale 23, il secondo 39. Nel dubbio, le parentesi rendono visibile quale operazione viene prima.

Prima di eseguire un'espressione controlliamo i tipi. `(x + 5) and (x > y)` può rispettare la grammatica, ma `and` richiede due booleani e `x + 5` è un intero. Il controllo dei tipi è detto **semantica statica**: analizza la forma senza eseguire il programma. Da qui in poi consideriamo espressioni correttamente tipate.

Scriviamo $\mathcal{E}(E, \sigma)$ per il valore dell'espressione E nello stato σ. Per calcolare `(x + 1) * y` nello stato dell'esempio, leggiamo $\sigma(x) = 10$, otteniamo $10 + 1 = 11$, leggiamo $\sigma(y) = 3$ e infine moltiplichiamo: $11 \cdot 3 = 33$. Lo stato non cambia.

La definizione generale segue la costruzione dell'espressione:

$$\mathcal{E}(x, \sigma) = \sigma(x), \qquad \mathcal{E}(E + E', \sigma) = \mathcal{E}(E, \sigma) + \mathcal{E}(E', \sigma).$$

Le costanti indicano i valori corrispondenti; `true` indica vero e `false` falso. Gli altri operatori si interpretano nello stesso modo, applicando l'operazione ai valori degli argomenti; le parentesi non cambiano il valore e `not` nega un booleano.

Qui `and` e `or` valutano entrambi gli argomenti: non assumiamo il cortocircuito di alcuni linguaggi reali.

5.1.1 Un'espressione può non avere valore

Il controllo dei tipi non impedisce ogni errore. `x div (y - 3)` richiede correttamente due interi; ma con $y = 3$ tenta una divisione per zero. Per alcune coppie espressione-stato il valore non esiste: perciò $\mathcal{E}$ è una **funzione parziale**. Indichiamo il valore mancante con $\perp$.

Precisiamo anche la divisione intera. Se $a \geq 0$ e $b > 0$, `a div b` è il quoziente e `a mod b` è il resto non negativo minore di b. Per esempio,

$$17 \text{ div } 5 = 3, \qquad 17 \text{ mod } 5 = 2.$$

Tutti gli esempi di divisione del libro usano questi segni. Per dare un significato anche agli altri casi, fissiamo, per qualsiasi intero a e qualsiasi $b \neq 0$, la convenzione

$$a \text{ div } b = \lfloor a/b \rfloor, \quad a \text{ mod } b = a - b\lfloor a/b \rfloor.$$

Il simbolo $\lfloor u \rfloor$ indica il più grande intero non maggiore di u. Questa scelta rende precisa la semantica del nostro linguaggio; non è una regola attribuita al C.

5.2 Assegnamento, sequenza e perdita di informazione

La scrittura $\sigma[v/x]$ indica lo stato uguale a σ su tutti i nomi tranne x, al quale associa v. Il comando `x := E` prima valuta tutta E nello stato vecchio, poi aggiorna x. Per esempio, da $x = 10$, $y = 3$, il comando `x := x + y` produce $x = 13$, $y = 3$.

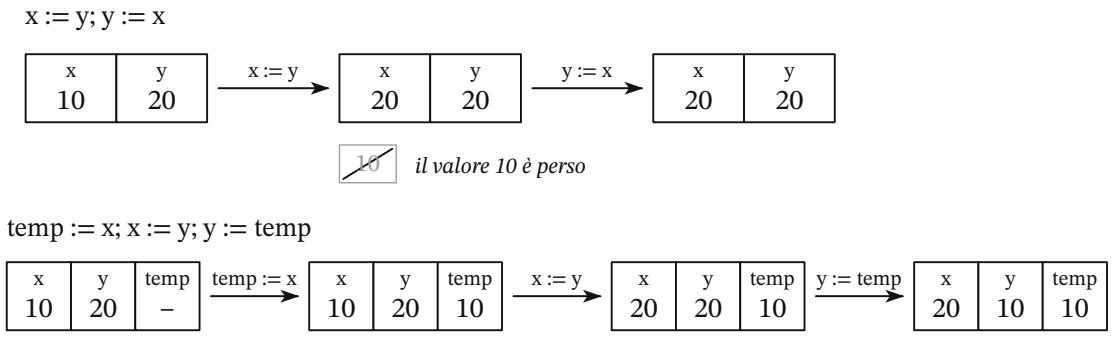
Concetto chiave · Espressioni e comandi
Un'espressione legge lo stato e calcola un valore. Un comando viene eseguito e, se termina con successo, produce uno stato. Un assegnamento sostituisce il vecchio valore della variabile destinazione.

Una sequenza `C; D` esegue prima `C`, poi `D` nello stato raggiunto. Per questo `x := y; y := x` non scambia i valori. Da $x = 10$, $y = 20$, il primo comando porta a $x = 20$, $y = 20$; il secondo legge il nuovo $x = 20$ e lascia ancora $y = 20$. Il vecchio 10 è stato perso.

Per conservarlo lo copiamo prima in una variabile ausiliaria:

```
temp := x;
x := y;
y := temp
```

Dopo il primo comando `temp` contiene 10; dopo il secondo `x` contiene 20; dopo il terzo `y` recupera 10. La necessità dell'appoggio nasce dal fatto che ogni comando legge lo stato corrente, non quello iniziale.



Lo scambio conserva temporaneamente il valore che il secondo assegnamento sovrascriverà. Le frecce seguono la direzione dell'esecuzione.

5.3 Scelte e ripetizioni: if e while

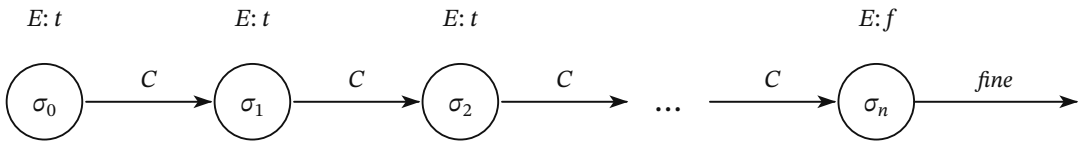
Il comando `skip` non fa nulla e lascia lo stato invariato. Il condizionale `if E then C else D fi` valuta la guardia booleana E. Se è vera esegue soltanto C; se è falsa esegue soltanto D. Se la guardia non è definita, non può scegliere il ramo.

Per ottenere il valore assoluto di `x` in `r` usiamo:

```
if x > 0 then r := x else r := -x fi
```

Con $x = 7$ scegliamo il primo ramo e otteniamo $r = 7$. Con $x = -7$ scegliamo il secondo e otteniamo ancora $r = 7$. Con $x = 0$ la guardia è falsa e il secondo ramo assegna $-0 = 0$. La variabile `x` resta invariata.

Il ciclo `while E do C endw` controlla la guardia prima di ogni eventuale esecuzione del corpo C. Se è falsa, esce subito. Se è vera, esegue il corpo e torna a controllare la guardia nello stato nuovo. Non decide in anticipo il numero di ripetizioni.



Ogni freccia rappresenta un'intera esecuzione del corpo. Il controllo della guardia avviene anche nello stato iniziale; il primo controllo falso conclude il ciclo.

Consideriamo una potenza, con base=2 ed esp=3:

```
ris := 1;
while esp > 0 do
    ris := ris * base;
    esp := esp - 1
endw
```

Al primo controllo abbiamo $(\text{ris}, \text{esp}) = (1, 3)$. Il corpo moltiplica il risultato per 2 e diminuisce l'esponente residuo: i controlli successivi incontrano $(2, 2)$, $(4, 1)$ e $(8, 0)$. Nell'ultimo stato la guardia è falsa, quindi il risultato è 8.

Se l'esponente iniziale è zero, il corpo non viene eseguito e rimane 1, come richiede $A^0 = 1$.

La traccia spiega questa esecuzione, ma non prova il risultato per ogni esponente. Per farlo cercheremo una relazione che leghi sempre il prodotto già calcolato al lavoro ancora da svolgere: sarà l'idea di invariante.

5.4 La semantica dei comandi e le esecuzioni senza risultato

Scriviamo $\mathcal{C}(C, \sigma)$ per lo stato finale prodotto da C a partire da σ. Per i primi costrutti:

$$\mathcal{C}(\text{skip}, \sigma) = \sigma,$$

$$\mathcal{C}(\text{x} := \text{E}, \sigma) = \sigma[\mathcal{E}(E, \sigma)/x],$$

$$\mathcal{C}(C; D, \sigma) = \mathcal{C}(D, \mathcal{C}(C, \sigma)).$$

La formula della sequenza rispecchia l'ordine dei comandi: lo stato prodotto da C viene passato a D. Per il condizionale si sceglie $\mathcal{C}(C, \sigma)$ oppure $\mathcal{C}(D, \sigma)$ secondo il valore della guardia.

Per il ciclo $W \equiv$ while E do C endw distinguiamo due casi. Se la guardia è falsa, non si esegue il corpo e lo stato resta quello iniziale:

$$\mathcal{C}(W, \sigma) = \sigma.$$

Se la guardia è vera, si esegue prima C e poi si ricomincia lo stesso ciclo nello stato ottenuto:

$$\mathcal{C}(W, \sigma) = \mathcal{C}(W, \mathcal{C}(C, \sigma)).$$

La seconda clausola richiama il ciclo stesso. Descrive il risultato quando le valutazioni necessarie sono definite e si raggiunge un controllo falso.

Anche $\mathcal{C}$ è parziale: non tutte le esecuzioni producono uno stato finale. Può verificarsi un errore, come in `x := 1 div 0`, oppure il calcolo può continuare per sempre.

Consideriamo

```
while x >= y do x := x + 1 endw
```

Se all'inizio $x \geq y$, ogni giro aumenta ancora x: la guardia resta sempre vera e il ciclo non termina. Se invece $x < y$, la guardia è subito falsa e si esce senza eseguire il corpo.

Il ciclo con `x := x - 1` al posto dell'incremento termina per ogni coppia di interi: y resta fisso e x diminuisce di uno finché diventa minore di y. La prova generale di questa affermazione richiederà una misura naturale del lavoro residuo, introdotta nel capitolo 7.

5.4.1 Tre modi di descrivere un programma

Quando seguiamo i singoli passi dell'esecuzione usiamo la **semantica operazionale**. Quando associamo a ogni comando la funzione parziale che porta dallo stato iniziale a quello finale usiamo la **semantica denotazionale**.

Nel capitolo seguente descriveremo il comportamento con proprietà vere prima e dopo i comandi. Le regole che collegano queste proprietà formano la **semantica assiomatica**.

Verifica 5

1. Esegui `x := x + y; y := x - y; x := x - y` da $x = 3, y = 7$. Poi usa valori iniziali generici A, B per spiegare il comportamento per tutti gli interi.

2. Per `while x ≠ 0 do x := x - 2 endw`, confronta le esecuzioni da $x = 4$ e da $x = 5$. Che cosa mostra l'esempio sulla necessità di controllare la terminazione?

Triple di Hoare e invarianti dei cicli

In breve
Come possiamo garantire il risultato per tutti gli ingressi ammessi? Sappiamo descrivere uno stato con una formula e seguire l'esecuzione dei comandi. Le triple di Hoare collegano ciò che chiediamo prima del programma a ciò che vogliamo sapere dopo.

Con $x = 17, y = 5$ possiamo seguire una divisione fino a ottenere quoziente 3 e resto 2. Per usare il programma su altri dati vogliamo di più: una prova che copra tutti gli ingressi ammessi.

6.1 Precondizione, postcondizione e correttezza

Un'**asserzione** è una formula sulle variabili del programma. Scriviamo $\sigma \vDash P$ per dire che P è vera nello stato σ.

Per esempio, con $\sigma(x) = 18$ e $\sigma(z) = -8$, la formula $x > 0 \land z < 0$ diventa $18 > 0 \land -8 < 0$ ed è vera. Indichiamo con P^σ la formula valutata usando i valori dello stato.

La verità dipende soltanto dalle variabili libere di P: cambiare un'altra variabile non può modificarla. Allo stesso modo, il valore di un'espressione dipende soltanto dai nomi che essa legge.

Concetto chiave · Tripla di Hoare
La tripla $\{P\}\ C\ \{Q\}$ collega una **precondizione** P, un comando C e una **postcondizione** Q.

In **correttezza totale**, ogni esecuzione da uno stato che soddisfa P termina con successo e soddisfa Q all'uscita.

In **correttezza parziale**, da uno stato che soddisfa P, ogni esecuzione che arriva a uno stato finale soddisfa Q.

La seconda nozione non garantisce che uno stato finale esista. Nella convenzione semantica qui adottata, errori ed esecuzioni infinite non producono uno stato finale: una prova soltanto parziale non esclude neppure gli errori. Manteniamo quindi espliciti i controlli di definitezza quando vogliamo un'esecuzione con successo. Scriveremo $\vDash_{tot}$ oppure $\vDash_{par}$ se occorre distinguere le due letture.

La tripla $\{x > 1\}\ \mathrm{x} := \mathrm{x} + 1\ \{x > 2\}$ è totalmente corretta: il nuovo valore è il precedente più uno, quindi supera 2. La tripla $\{x = 5\}\ \mathrm{x} := \mathrm{x} + 1\ \{x = 5\}$ è falsa: l'unico valore finale possibile di x è 6.

Possiamo usare una tripla per descrivere un comando, cercando Q; per costruire un programma, cercando C; oppure per verificare una proposta, avendo già tutti e tre gli elementi.

6.1.1 Il valore finale e quello iniziale

Una postcondizione va letta nello stato finale. Così $\{x > 0\}\ C\ \{y = 2x\}$ ammette anche `y := 10; x := 5`.

Se desideriamo il doppio dell'ingresso, dobbiamo fissarlo con una variabile di specifica: $\{x = A \land A > 0\}\ C\ \{y = 2A\}$. La lettera A resta fissata durante la prova e non compare nei comandi.

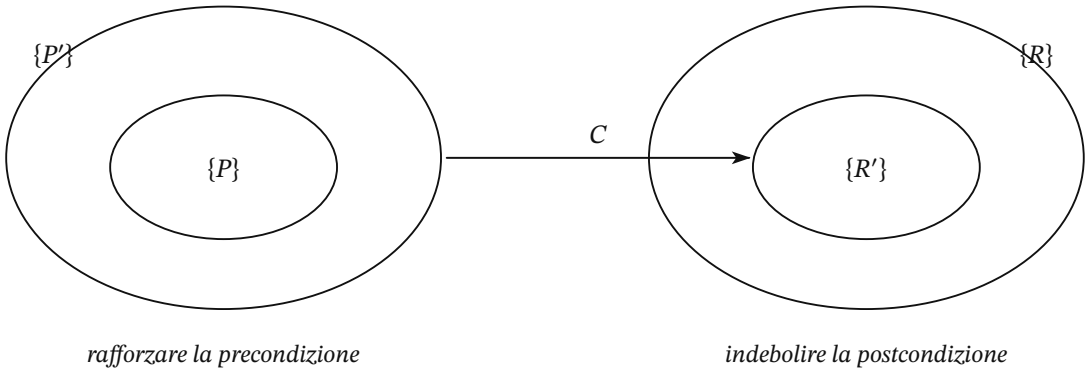
6.2 Conseguenza e definitezza delle espressioni

Se un programma funziona su tutti gli stati con $x > 0$, funziona anche su quelli con $x > 10$, che sono meno. È il **rafforzamento della precondizione**.

Se garantisce $x = 5$, garantisce anche $x > 0$, che chiede meno: è l'**indebolimento della postcondizione**. La regola di conseguenza riunisce queste osservazioni:

$$\frac{P \Rightarrow P' \quad \{P'\}\,C\,\{Q'\} \quad Q' \Rightarrow Q}{\{P\}\,C\,\{Q\}}.$$

Le implicazioni sono richieste per tutti gli stati considerati. In termini di insiemi, gli stati di P sono contenuti in quelli di P'; gli stati di Q' sono contenuti in quelli di Q. Non possiamo allargare arbitrariamente gli ingressi o pretendere conclusioni più forti.



Una precondizione più forte restringe i punti di partenza; una postcondizione più debole amplia i risultati ammessi.

Per garantire che un'espressione E abbia valore introduciamo l'asserzione $\mathrm{def}(E)$. Le costanti e le variabili presenti nello stato sono definite. Per somme, prodotti, differenze, confronti e operatori booleani occorre che siano definiti gli argomenti. Per divisione e resto occorre anche il divisore non nullo:

$$\mathrm{def}(E \text{ div } D) \equiv \mathrm{def}(E) \wedge \mathrm{def}(D) \wedge D \neq 0.$$

La stessa clausola vale per `mod`. Per negazione e parentesi basta la definitezza dell'espressione interna. Per esempio, $\mathrm{def}(x \text{ div } (z - y))$ equivale a $z \neq y$: la sottrazione è definita, ma il suo risultato non deve essere zero. Con valutazione senza cortocircuito, `x ≠ 0 and y div x > 1` richiede comunque $x \neq 0$ per essere definita.

6.3 Assegnamento: ragionare a partire dal risultato

Vogliamo garantire $x > y$ dopo `x := x + 2`. Il valore finale di x sarà il vecchio $x + 2$. Dobbiamo dunque chiedere, prima, $x + 2 > y$. Non stiamo sostituendo il programma con una formula: stiamo descrivendo la condizione che il suo risultato deve soddisfare.

In generale $Q[E\,/\,x]$ sostituisce E alle occorrenze libere di x in Q, senza catturare variabili. Rinominiamo prima eventuali variabili quantificate che ostacolano la sostituzione. Se E è definita, vale la proprietà fondamentale:

$$\sigma[\mathcal{E}(E, \sigma)\,/\,x] \vDash Q \text{ se e solo se } \sigma \vDash Q[E\,/\,x].$$

A sinistra valutiamo Q dopo l'aggiornamento; a destra anticipiamo nella formula il valore che verrà assegnato. Questa proprietà giustifica l'assioma dell'assegnamento:

$$\{\mathrm{def}(E) \wedge Q[E\,/\,x]\} \;\; \mathrm{x} := \mathrm{E} \;\; \{Q\}.$$

La definitezza assicura che il comando produca uno stato; la sostituzione assicura che quel risultato soddisfi Q.

Se abbiamo già una precondizione P, dobbiamo dimostrare $P \Rightarrow \mathrm{def}(E) \wedge Q[E/x]$. La regola di conseguenza permette poi di usare P come ingresso.

Per `x := 5` e postcondizione $x = 5$, la sostituzione dà $5 = 5$, sempre vera. Non serve alcuna richiesta sul vecchio valore di x.

Per `x := 1 div y`, invece, serve $y \neq 0$ anche se la postcondizione non legge x: in correttezza totale dobbiamo comunque escludere l'errore.

Il comando `skip` ha l'assioma $\{P\}$ skip $\{P\}$: nessun valore cambia, quindi ogni asserzione vera prima resta vera dopo.

6.4 Sequenza: trovare l'asserzione intermedia

Per provare $\{P\}\ C; D\ \{Q\}$ cerchiamo un'asserzione R tale che $\{P\}C\{R\}$ e $\{R\}D\{Q\}$. Il primo comando consegna al secondo uno stato nelle condizioni che gli servono. È la regola della sequenza.

Riprendiamo lo scambio, con $x = N \wedge y = M$ all'ingresso e $x = M \wedge y = N$ all'uscita. Il programma ha la forma `t := E; x := y; y := t`. Quale valore deve salvare il primo comando?

Partiamo dall'ultimo assegnamento. Dopo `y := t` vogliamo $y = N$, quindi prima ci serve $t = N$. La richiesta su x resta la stessa, perché il comando non lo cambia.

Risaliamo poi attraverso `x := y`. Per ottenere $x = M$ dopo questo passo, prima deve valere $y = M$. Le condizioni prima dei tre comandi sono dunque:

Comando	Condizione prima del comando
y := t	$x = M \wedge t = N$
x := y	$y = M \wedge t = N$
t := E	$y = M \wedge E = N$

Il primo comando deve anche avere un'espressione definita. L'ingresso offre $x = N$: scegliamo $E = x$. Leggiamo la variabile di programma `x`; N serve soltanto alla prova e non può comparire nei comandi.

Con $N = 3, M = 7$ l'esecuzione va da $(x, y) = (3, 7)$ a $(7, 3)$, passando per $t = 3$. L'esecuzione procede dall'alto verso il basso nel programma; la ricerca delle condizioni è partita invece dall'ultimo comando.

Un esempio più breve è `x := x + 1; y := y - 1`, con risultato $x > y$. Prima dell'ultimo comando serve $x > y - 1$; prima del primo serve $x + 1 > y - 1$. La precondizione $x \geq y - 1$ è sufficiente, perché dà $x + 1 \geq y > y - 1$. La regola non esige che la precondizione data sia esattamente quella calcolata: basta che la implichi.

6.5 Condizionale: utilizzare l'informazione della guardia

Per verificare `if E then C else D fi` con ingresso P e uscita Q controlliamo che $P \Rightarrow \mathrm{def}(E)$. Poi dimostriamo $\{P \wedge E\}C\{Q\}$ e $\{P \wedge \neg E\}D\{Q\}$. Ogni ramo riceve l'informazione che ha determinato la scelta.

Vogliamo che m contenga il massimo di x, y. Usiamo

```
if x < y then m := y else m := x fi
```

La guardia è definita. Nel primo ramo sappiamo $x < y$: assegnare y a m dà il massimo. Nel secondo sappiamo $x \geq y$: assegnare x dà ancora il massimo.

Non serve conoscere il vecchio valore di m, perché entrambi i rami lo sostituiscono.

6.5.1 Una guardia che richiede una divisione

Consideriamo una scelta che usa la divisione:

```
if x div y > 0 then m := x else m := y fi
```

L'ipotesi $x \geq 0 \wedge y \geq 0$ non basta: permette $y = 0$, con cui la guardia non ha valore. Chiediamo invece $x \geq 0 \wedge y > 0$.

Con questi vincoli, x div $y > 0$ equivale a $x \geq y$. Nel ramo vero scegliamo quindi il massimo x; nel ramo falso scegliamo il massimo y.

L'ipotesi sul divisore ha due ruoli: rende possibile la valutazione e permette di leggere correttamente il confronto.

6.6 Invariante e correttezza parziale del while

Passiamo alla divisione mediante sottrazioni, con $x \geq 0$, $y > 0$ e x, y non modificati:

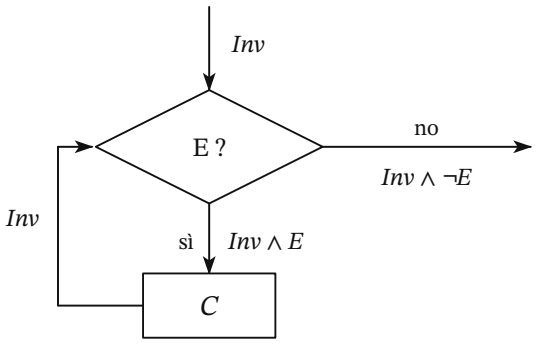
```
q := 0; r := x;
while r >= y do
    q := q + 1;
    r := r - y
endw
```

Da $x = 17, y = 5$ otteniamo ai controlli le coppie $(q, r) = (0,17), (1,12), (2,7), (3,2)$. Che cosa collega queste righe? Ogni volta aumenta di 5 la quantità $q\,y$ e diminuisce di 5 il resto r. Si conserva $x = q\,y + r$.

Inoltre $r \geq 0$: il corpo sottrae y solo quando $r \geq y$.

Scegliamo perciò $I \equiv (x = q\,y + r \wedge r \geq 0 \wedge y > 0)$. È un **invariante**: vale dopo l'inizializzazione e viene ripristinato dopo ogni intera esecuzione del corpo.

Dopo il solo incremento di q l'uguaglianza può essere falsa; torna vera dopo la sottrazione da r. Il punto in cui la richiediamo è il controllo della guardia.



La proprietà deve valere prima del controllo e dopo l'intero corpo. All'uscita aggiungiamo l'informazione che la guardia è falsa.

6.6.1 Perché l'invariante vale anche all'uscita

La regola generale, in correttezza parziale, è

$$\frac{\{I \wedge E\}\ C\ \{I\}}{\{I\}\ \text{while E do C endw}\ \{I \wedge \neg E\}}.$$

Perché funziona? Prima della prima iterazione vale I. Se vale prima di un giro eseguito, è vera anche E e la premessa ripristina I. Ripetendo questo ragionamento per il numero di giri effettivamente completati, I vale a ogni controllo raggiunto.

Se il ciclo produce uno stato finale, in quello stato la guardia è falsa: otteniamo $I \wedge \neg E$. È un ragionamento per induzione sul numero di iterazioni, senza dover conoscere quel numero in anticipo.

Per una specifica generale servono anche ingresso e uscita: $P \Rightarrow I$ e $I \wedge \neg E \Rightarrow Q$. Nella divisione la guardia falsa dà $r < y$; insieme all'invariante otteniamo $x = q\,y + r \wedge 0 \le r < y$. La prova descrive il risultato di ogni esecuzione che arriva all'uscita. Nel prossimo capitolo dimostriamo che quell'uscita viene raggiunta con successo.

VERIFICA 6

1. Calcola una precondizione che garantisca $y > 10$ dopo `y := x * 2`. Verifica poi se $x \ge 6$ è sufficiente. Motiva con l'assioma, senza limitarti a provare alcuni valori.

2. La precondizione $z = 5 \wedge x = 3$ garantisce $z > 4$ dopo `x := 0; y := z div x` in correttezza totale? Calcola all'indietro il controllo di definitezza e spiega la risposta.

Correttezza totale e funzioni di terminazione

In breve
Come dimostriamo che un ciclo arriva all'uscita? Nella divisione abbiamo visto che il resto diminuisce. Cercheremo una quantità naturale che scenda a ogni giro e useremo questa idea per completare le prove di correttezza.

Nella divisione ogni sottrazione riduce il resto. Questo fatto suggerisce perché il ciclo debba finire, ma dobbiamo precisare che cosa diminuisce e perché non può diminuire per sempre. L'invariante descrive la relazione che vogliamo conservare; una funzione di terminazione descrive il progresso verso l'uscita.

7.1 Una quantità naturale che diminuisce

Con $x = 17$, $y = 5$, il resto ai controlli è $17, 12, 7, 2$. È sempre un intero non negativo e diminuisce strettamente a ogni giro eseguito. Una sequenza di questo genere non può essere infinita: partendo da un naturale k, dopo ogni diminuzione intera si perde almeno uno, quindi non si possono compiere più di k diminuzioni senza scendere sotto zero.

Concetto chiave · Funzione di terminazione
Una funzione di terminazione associa agli stati considerati una quantità t intera, definita e non negativa. Se ogni iterazione completata la fa diminuire strettamente, e ogni iterazione termina con successo, il ciclo non può eseguire infinite iterazioni.

7.1.1 Perché chiediamo un numero naturale

La sola diminuzione non basta. La successione di interi $0, -1, -2, \ldots$ diminuisce senza fermarsi; serve il limite inferiore. Neppure essere reale e positivo basta: $1, 1/2, 1/4, \ldots$ è una successione infinita strettamente decrescente. La scelta dei naturali contiene quindi una parte essenziale dell'argomento.

7.1.2 Confrontare il valore vecchio e quello nuovo

Per confrontare il valore prima e dopo il corpo usiamo una variabile di specifica fresca V. Nella premessa $t = V$ essa memorizza il valore di partenza; nessun comando può modificarla. Il progresso si scrive

$$\{I \wedge E \wedge t = V\}\ C\ \{t < V\}.$$

Non significa che il programma debba conservare t. Significa che la prova può confrontare il nuovo valore di t con il suo valore vecchio, rimasto fissato in V.

7.2 La regola del while in correttezza totale

Una prova totale deve garantire tre fatti: ogni controllo della guardia ha valore; ogni corpo che viene iniziato termina con successo; il numero di iterazioni è finito. Ecco gli obblighi per $\{P\}\ W\ \{Q\}$, dove W è `while E do C endw`:

1. **Ingresso.** $P \Rightarrow I \wedge \mathrm{def}(E)$: prima del primo controllo valgono l'invariante e la definitezza della guardia.
2. **Preservazione.** $\{I \wedge E\}C\{I \wedge \mathrm{def}(E)\}$ in correttezza totale: il corpo termina con successo e prepara il controllo successivo.
3. **Limite inferiore.** $I \Rightarrow t \geq 0$, con t intera e definita negli stati di I.
4. **Progresso.** $\{I \wedge E \wedge t = V\}C\{t < V\}$ in correttezza totale, per ogni valore fissato di V.
5. **Uscita.** $I \wedge \neg E \Rightarrow Q$: quando il ciclo si ferma, le informazioni rimaste danno il risultato richiesto.

La preservazione garantisce che il ragionamento possa continuare a ogni controllo. Se la guardia rimanesse sempre vera, otterremmo infinite diminuzioni fra naturali, impossibili. Dunque incontriamo un controllo falso; l'invariante e l'uscita danno Q.

Quando un'inizializzazione precede il ciclo, il primo obbligo è una tripla sul comando iniziale, anziché un'implicazione diretta.

7.3 Divisione intera: completare la prova

Riprendiamo `q := 0; r := x` e il corpo `q := q + 1; r := r - y`, con guardia $r \geq y$. L'invariante è $I \equiv (x = qy + r \wedge r \geq 0 \wedge y > 0)$ e scegliamo $t = r$. La positività di y compare nell'invariante perché la useremo per dimostrare il progresso a ogni giro.

7.3.1 Inizializzazione e conservazione

L'inizializzazione stabilisce $x = 0 \cdot y + x$ e $r = x \geq 0$. La guardia contiene solo un confronto fra variabili e quindi è definita.

Per verificare che l'invariante si conservi, indichiamo con q, r i valori prima del corpo e con q', r' quelli dopo. La sequenza produce $q' = q + 1$ e $r' = r - y$. Perciò

$$q'y + r' = (q+1)y + (r-y) = qy + r = x.$$

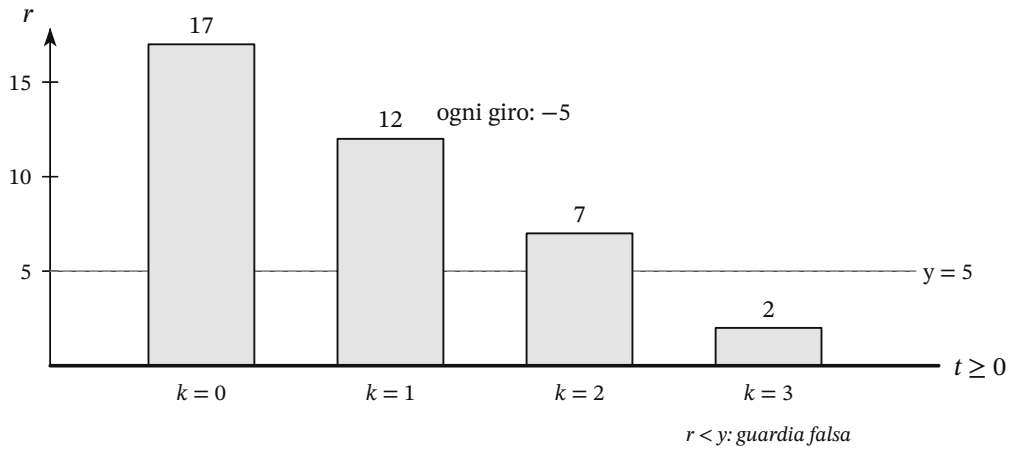
La guardia $r \geq y$ assicura $r' \geq 0$. Il corpo non assegna a y, quindi $y > 0$ resta vera. Somma e sottrazione sono definite e terminano: abbiamo provato la conservazione dell'invariante con un'esecuzione che termina con successo.

7.3.2 Progresso e risultato

Dall'invariante abbiamo $t = r \geq 0$. Se prima del corpo $r = V$, dopo vale $r' = V - y < V$, perché $y > 0$. Abbiamo così la diminuzione stretta. All'uscita la guardia falsa dà $r < y$; insieme all'invariante otteniamo

$$x = qy + r \wedge 0 \leq r < y.$$

Questa è precisamente la caratterizzazione del quoziente e del resto della divisione di x per y. Il programma termina e li calcola per ogni $x \geq 0, y > 0$. Con $y = 0$, invece, la guardia sarebbe sempre vera e la sottrazione non cambierebbe r: fallirebbe il progresso. L'ipotesi sul divisore ha dunque un motivo concreto.



Per $x = 17, y = 5$ il resto scende da 17 a 2. Il limite inferiore e la diminuzione intera spiegano la terminazione; l'invariante spiega il significato del risultato.

7.4 Somma dei primi interi: scegliere l'invariante

Vogliamo sommare gli interi da 0 a $n-1$, con $n \geq 0$ non modificato. Invece di memorizzare tutti gli addendi, teniamo la somma degli addendi già attraversati e l'indice del prossimo:

```
x := 0; s := 0;
while x < n do
    s := s + x;
    x := x + 1
endw
```

Prima del controllo, x è il prossimo intero da aggiungere. Gli addendi già inclusi sono dunque 0, $1, \ldots, x-1$. Questa lettura suggerisce

$$I \equiv s = \sum_{i=0}^{x-1} i \wedge 0 \leq x \leq n.$$

La somma è vuota e vale zero quando $x = 0$. L'inizializzazione è quindi corretta. I limiti su x servono sia a interpretare la parte già percorsa sia a riconoscere il punto finale.

7.4.1 Preparare il giro successivo

Per conservare l'invariante chiediamoci che cosa deve valere prima dell'ultimo comando `x := x + 1`. Sostituendo $x+1$ a x nell'invariante otteniamo

$$s = \sum_{i=0}^{x} i \wedge 0 \leq x+1 \leq n.$$

Prima di `s := s + x` serve quindi

$$s + x = \sum_{i=0}^{x} i \wedge 0 \leq x+1 \leq n.$$

La prima uguaglianza viene dall'invariante e dall'identità

$$\sum_{i=0}^{x} i = \sum_{i=0}^{x-1} i + x.$$

La guardia $x < n$, insieme a $0 \le x$ e al fatto che x, n sono interi, dà $0 \le x+1 \le n$. Abbiamo verificato entrambe le richieste; tutte le espressioni sono definite.

7.4.2 Quanto lavoro rimane?

Per la terminazione prendiamo $t = n - x$, il numero di addendi ancora da attraversare. Da $x \le n$ segue $t \ge 0$. Un giro aumenta x di uno e lascia n fisso, quindi il nuovo valore è $V - 1 < V$. All'uscita $x \ge n$ e $x \le n$ danno $x = n$: l'invariante diventa la somma richiesta. Per $n = 0$ non eseguiamo alcun giro e otteniamo correttamente zero.

È essenziale l'ordine degli assegnamenti: aggiungiamo il vecchio x, poi lo incrementiamo. Scambiandoli sommeremmo gli interi da 1 a n, risolvendo un problema diverso.

7.5 Programmi annotati e proprietà non modificate

Annotare un programma significa scrivere asserzioni nei punti utili alla prova. Per la somma possiamo indicare I e $t = n - x$ prima del `while`, $I \wedge x < n$ all'ingresso del corpo, l'asserzione intermedia appena ricavata fra i due assegnamenti e $I \wedge x \ge n$ dopo il ciclo. Queste scritte non vengono eseguite: dichiarano gli obblighi che il lettore deve giustificare.

Spesso serve trasportare un'informazione che il comando non tocca. Sia $\mathrm{change}(C)$ l'insieme dei nomi che compaiono come destinazione di un assegnamento in C. Sono nomi che il comando potrebbe modificare, anche se un particolare ramo non viene eseguito.

Con $\mathrm{free}(R)$ indichiamo le variabili libere dell'asserzione R. Se

$$\mathrm{free}(R) \cap \mathrm{change}(C) = \varnothing,$$

la verità di R non cambia durante un'esecuzione con successo di C. Dalla tripla $\{P\}\,C\,\{Q\}$ possiamo dunque ricavare $\{R \wedge P\}\,C\,\{R \wedge Q\}$. La condizione sulle variabili è ciò che giustifica il passaggio. Nella divisione il corpo assegna solo a q, r, quindi conserva $y > 0$.

7.5.1 Combinare due garanzie

Possiamo anche combinare prove dello stesso comando. Da $\{P_1\}\,C\,\{Q_1\}$ e $\{P_2\}\,C\,\{Q_2\}$ ricaviamo $\{P_1 \wedge P_2\}\,C\,\{Q_1 \wedge Q_2\}$: uno stato che soddisfa entrambi gli ingressi gode di entrambe le garanzie sul risultato.

Da $\{P\}\,C\,\{Q\}$ e $\{R\}\,C\,\{Q\}$ ricaviamo $\{P \vee R\}\,C\,\{Q\}$: distinguiamo quale ingresso vale e applichiamo la prova corrispondente. In questo linguaggio deterministico la stessa esecuzione finale permette di congiungere le conclusioni.

7.5.2 Per proseguire: eliminare gli incroci

Un esempio geometrico mostra che la funzione di terminazione va scelta pensando al problema, non soltanto al codice. Sono fissati N punti blu e N punti rossi distinti, senza tre punti allineati. Vogliamo abbinarli con segmenti fra colori diversi, senza incroci.

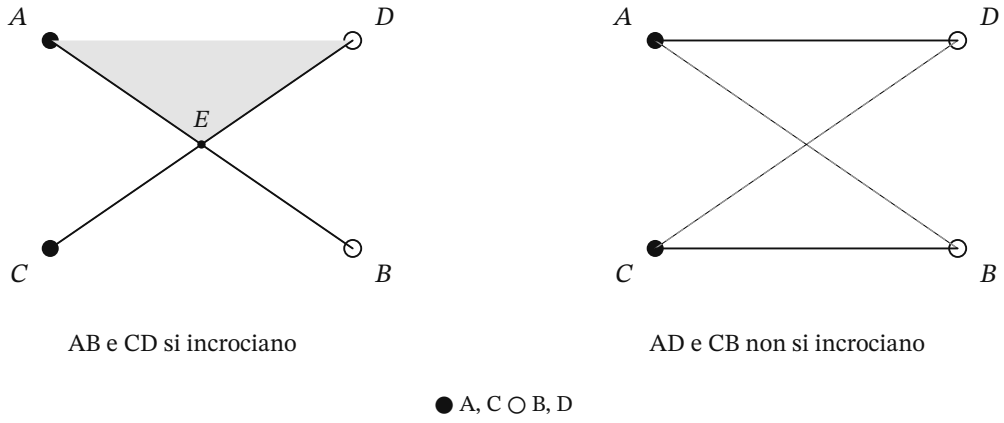
Partiamo da un abbinamento qualsiasi; finché due segmenti si incrociano, sostituiamoli con i due collegamenti non incrociati che rispettano i colori.

L'invariante afferma che ogni punto appartiene esattamente a un segmento e ogni segmento unisce colori diversi. Lo scambio lo conserva: ricongiunge gli stessi quattro estremi, ciascuno una sola volta. All'uscita non ci sono incroci, quindi l'obiettivo è raggiunto, se il procedimento termina.

Contare gli incroci non offre direttamente una prova di progresso: togliere l'incrocio scelto può crearne con altri segmenti. Consideriamo invece la somma delle lunghezze. Se $A\,B$ e $C\,D$ si incontrano in E e i colori consentono i nuovi segmenti $A\,D, C\,B$, la disuguaglianza triangolare dà

$$\ell(A\,D) < \ell(A\,E) + \ell(E\,D), \quad \ell(C\,B) < \ell(C\,E) + \ell(E\,B).$$

Le disuguaglianze sono strette: i triangoli coinvolti non sono degeneri, per l'ipotesi di non allineamento. Sommando, la nuova lunghezza totale è strettamente minore di quella vecchia. Gli altri segmenti restano invariati.



I percorsi spezzati che passano per E sono più lunghi dei nuovi collegamenti rettilinei. Il risparmio riguarda la lunghezza totale.

Le lunghezze sono reali, quindi la sola positività non basterebbe. Qui però i punti sono fissati e gli abbinamenti possibili sono finiti: $N!$, perché a ciascun blu associamo un diverso rosso. Anche i valori della lunghezza totale sono finiti in numero.

Una diminuzione stretta impedisce di tornare a un valore precedente e quindi non può continuare all'infinito. Questa è una prova completa di terminazione basata sulla finitezza dello spazio degli stati possibili.

Verifica 7

1. Per `while i < n do i := i + 1 endw`, con $I \equiv 0 \le i \le n$ e n fisso, proponi una funzione di terminazione e verifica limite inferiore, progresso e uscita.

2. Nel programma della somma scambia i due assegnamenti del corpo. Trova un invariante che descriva la somma ora calcolata e spiega il risultato finale, compreso il caso $n = 0$.

Costruire un ciclo: il significato delle variabili

In breve
Come troviamo un ciclo, prima ancora di verificarlo? Partiamo dal risultato desiderato e da esempi piccoli. Sceglieremo che cosa devono ricordare le variabili durante il calcolo, poi costruiremo i comandi che mantengono quel significato.

8.1 Dalla postcondizione a una proprietà da conservare

Per ottenere il quoziente della divisione di 17 per 5, possiamo sottrarre ripetutamente 5 e contare le sottrazioni. Dopo due passi abbiamo tolto 10 e rimane 7: l'uguaglianza $17 = 2 \cdot 5 + 7$ racconta dove siamo arrivati. Il resto non è ancora piccolo abbastanza, ma l'uguaglianza è già vera.

Questo spiega la scelta fatta nel capitolo precedente: separare la relazione da conservare, $x = q\,y + r \wedge r \geq 0$, dalla condizione da raggiungere, $r < y$. La prima diventa un invariante, insieme al vincolo $y > 0$; la seconda suggerisce la guardia opposta, `r >= y`.

Introdurre r rende visibile il testimone dell'esistenziale che caratterizza il quoziente: esiste un resto con $x = q\,y + r$ e $0 \leq r < y$.

In generale cerchiamo due proprietà I e G tali che $I \wedge G$ implichi la postcondizione. Vogliamo conservare I e continuare finché G è falsa. Occorrono poi un'inizializzazione che stabilisca I, un corpo che la ristabilisca e una prova di terminazione. La guardia deve essere un'espressione del linguaggio, quindi non ogni scomposizione matematica è utilizzabile.

Concetto chiave · Invariante come significato delle variabili
Un buon invariante spiega che cosa rappresentano i valori presenti durante il calcolo. Si rafforza con i vincoli necessari per leggere le espressioni, eseguire il passo successivo e dimostrare la terminazione.

8.2 Fattoriale: ricordare il prodotto già calcolato

Il fattoriale $n!$ è il prodotto da 1 a n; per convenzione $0! = 1$. Supponiamo $n \geq 0$ intero e non modificato. Per $n = 4$ vogliamo ottenere in successione 1, $1 \cdot 1$, $1 \cdot 1 \cdot 2$, $1 \cdot 1 \cdot 2 \cdot 3$, $1 \cdot 1 \cdot 2 \cdot 3 \cdot 4$. Serve una variabile per il prodotto e una per il prossimo fattore.

```
fatt := 1; x := 1;
while x <= n do
    fatt := fatt * x;
    x := x + 1
endw
```

Prima di usare il fattore x, sono già stati moltiplicati gli interi fino a $x - 1$. Cerchiamo dunque

$$I \equiv \text{fatt}{=}(x-1)! \wedge 1 \leq x \leq n+1.$$

Il limite inferiore 1 assicura che il fattoriale nell'asserzione abbia un argomento naturale. Il limite superiore $n+1$ include lo stato finale: dopo aver usato n, il contatore avanza ancora una volta.

8.2.1 Controllare il significato del prodotto

All'inizio $x = 1$ e fatt=$1 = 0!$; inoltre $1 \leq n + 1$.

Supponiamo ora che I valga e che $x \leq n$. La prima assegnazione produce fatt' $= (x-1)! \cdot x = x!$. La seconda porta x a $x' = x + 1$. La relazione da ritrovare è fatt' $= (x' - 1)!$, ed è proprio l'uguaglianza appena ottenuta.

Anche i limiti si conservano: $1 \leq x' \leq n + 1$.

8.2.2 Fermarsi dopo l'ultimo fattore

La quantità $n + 1 - x$ è un intero non negativo sotto I e diminuisce di 1 a ogni giro. Tutte le espressioni del corpo e della guardia sono definite. Il ciclo termina.

All'uscita $x > n$; insieme a $x \leq n + 1$ e al fatto che x, n sono interi, questo dà $x = n + 1$. Dunque fatt=$n!$.

Per $n = 0$ non si entra nel ciclo e si conserva il risultato 1. La funzione $n - x$ scenderebbe invece a -1 nello stato finale: per la regola adottata nel corso, che chiede non negatività sotto tutto l'invariante, scegliamo $n + 1 - x$.

8.3 MCD: trasformare i dati conservando la risposta

Il massimo comune divisore di 18 e 12 è anche il massimo comune divisore di 6 e 12. Infatti un intero divide 18 e 12 se e solo se divide $18 - 12$ e 12: una direzione usa la differenza, l'altra la somma $18 = (18 - 12) + 12$. Sottrarre il minore dal maggiore conserva quindi tutti i divisori comuni.

Con $a, b > 0$ interi e non modificati, lavoriamo su due copie:

```
x := a; y := b;
while x != y do
    if x > y
        then x := x - y
        else y := y - x
    fi
endw
```

Il risultato da conservare non è una somma già accumulata: è il MCD dei due valori di lavoro. Scegliamo

$$I \equiv x > 0 \wedge y > 0 \wedge \mathrm{mcd}(x, y) = \mathrm{mcd}(a, b).$$

L'inizializzazione rende vera questa proprietà. Se $x > y$, allora $x - y > 0$ e la proprietà dei divisori comuni appena dimostrata conserva il MCD.

Se si esegue l'altro ramo, sappiamo sia $x \neq y$ sia $\neg(x > y)$: perciò $x < y$, e vale il ragionamento simmetrico. La guardia è importante anche per giustificare questo secondo caso.

8.3.1 Una somma che diminuisce

Come misurare il lavoro che rimane? Non serve sapere quanti passi occorreranno: basta una quantità che scenda. $x + y$ è positiva sotto I. Nel primo ramo il suo nuovo valore è $(x - y) + y = x$, strettamente minore del vecchio $x + y$; nel secondo è y, ancora minore. Quindi il ciclo termina.

All'uscita $x = y > 0$, e il MCD di due numeri positivi uguali è quel numero. L'invariante dà $x = \mathrm{mcd}(a, b)$. Per esempio, da $(x, y) = (12, 18)$ passiamo a $(12, 6)$ e poi a $(6, 6)$.

8.4 Radice cubica: costruire potenze con differenze

Dato un intero $x \geq 0$, vogliamo l'intero $r \geq 0$ per cui $r^3 \leq x < (r+1)^3$. Con $x = 10$, i cubi sono 0, $1, 8, 27$: la risposta è 2, perché $8 \leq 10 < 27$.

Proviamo i cubi in ordine, fino al primo che supera x. Per passare da un cubo al successivo usando soltanto somme sfruttiamo

$$(b+1)^3 = b^3 + 3b^2 + 3b + 1.$$

Il solo cubo non basta: ci serve anche il quadrato. Conserviamo perciò l'indice in b, il quadrato in a e il cubo in y. Per aggiornare il quadrato usiamo $(b+1)^2 = b^2 + 2b + 1$.

```
a := 0; b := 0; y := 0;
while y <= x do
    y := y + 3*a + 3*b + 1;
    a := a + 2*b + 1;
    b := b + 1
endw
```

L'ordine conta: `y` legge i vecchi `a` e `b`; poi `a` legge il vecchio `b`; soltanto alla fine aumentiamo `b`.

8.4.1 Seguire il calcolo per 10

Ai controlli della guardia otteniamo:

b	$a = b^2$	$y = b^3$	$y \leq 10$
0	0	0	vero
1	1	1	vero
2	4	8	vero
3	9	27	falso

Guardiamo l'ultimo giro. Si parte da $a = 4, b = 2, y = 8$. Il primo comando calcola $8 + 3 \cdot 4 + 3 \cdot 2 + 1 = 27$; il secondo calcola $4 + 2 \cdot 2 + 1 = 9$; il terzo porta b a 3. Al controllo successivo usciamo.

La variabile b ha superato la risposta di uno. Il cubo attuale supera 10, ma il precedente, $2^3 = 8$, era ancora ammesso. Questo è il fatto da ricordare nell'invariante.

8.4.2 Ricordare anche il cubo precedente

Ai controlli vogliamo conservare

$$I \equiv a = b^2 \wedge y = b^3 \wedge (b-1)^3 \leq x \wedge b \geq 0 \wedge x \geq 0.$$

All'inizio $a = b = y = 0$. La condizione sul cubo precedente vale perché $(b-1)^3 = -1 \leq x$. Non indica un cubo già provato dal programma: permette soltanto di usare la stessa formula anche nello stato iniziale.

Supponiamo vera la guardia. Le due identità sulle potenze assicurano che i nuovi valori siano il quadrato e il cubo del nuovo indice $b' = b + 1$.

Resta il cubo precedente. Prima del giro sappiamo $b^3 = y \leq x$, grazie alla guardia. Dopo il giro il cubo precedente è proprio $(b'-1)^3 = b^3$. Anche questa parte di I è quindi conservata; i limiti $b' \geq 0, x \geq 0$ rimangono veri.

8.4.3 Come trovare una quantità che diminuisce

Il cubo y cresce. Verrebbe da misurare quanto manca a x con $x - y$, ma nell'ultimo stato questa quantità è negativa: per 10 diventerebbe $10 - 27 = -17$. Non rispetta la non negatività richiesta sotto tutto l'invariante.

L'invariante ci offre un'altra quantità: il cubo *precedente* non supera mai x. La distanza $x - (b-1)^3$ è quindi sempre non negativa. Aggiungiamo 1 e scegliamo

$$t = x + 1 - (b-1)^3.$$

Così $t \geq 1$ in ogni stato di I, compreso quello finale. Con $x = 10$ possiamo controllare la scelta sulle stesse quattro righe:

b	$(b-1)^3$	$t = 11 - (b-1)^3$
0	-1	12
1	0	11
2	1	10
3	8	3

La traccia suggerisce la diminuzione; ora la dimostriamo per ogni giro. Dopo il corpo $b' = b + 1$, quindi $t' = x + 1 - b^3$. Sottraendo troviamo

$$t - t' = b^3 - (b-1)^3 = 3b(b-1) + 1.$$

Per $b = 0$ e $b = 1$ il risultato è 1. Per ogni intero $b \geq 2$, il prodotto $b(b-1)$ è positivo. Dunque $t - t' > 0$ per tutti i valori ammessi: t scende strettamente.

La guardia e il corpo usano soltanto operazioni sempre definite sugli interi. Ogni giro termina con successo; la funzione naturale t esclude infiniti giri.

8.4.4 Leggere il risultato all'uscita

Quando usciamo, $y > x$. Dall'invariante otteniamo

$$(b-1)^3 \leq x < b^3.$$

All'inizio la guardia è vera, perché $0 \leq x$. È stato quindi eseguito almeno un giro e all'uscita $b \geq 1$. La risposta $b - 1$ è non negativa ed è proprio la parte intera della radice cubica.

> **Attenzione · Quale variabile contiene la radice?**
> Il ciclo lascia in `b` un valore in più: per $x = 0$ lascia 1, per $x = 8$ lascia 3. Per consegnare la risposta in `b`, aggiungiamo dopo il ciclo `b := b - 1`. L'invariante riguarda il ciclo prima di questa correzione.

8.5 Fibonacci: conservare il valore che manca

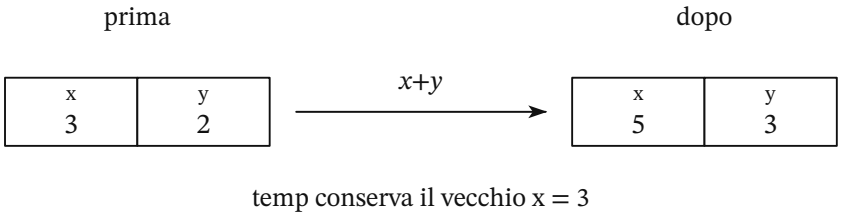
La successione di Fibonacci parte da $\text{fib}(0) = 0$ e $\text{fib}(1) = 1$; ogni termine successivo è la somma dei due precedenti. Per calcolare $\text{fib}(k)$ con $k \geq 1$, vorremmo mantenere $x = \text{fib}(i)$ e far avanzare i fino a k.

Il prossimo valore dovrebbe essere $\text{fib}(i+1) = x + \text{fib}(i-1)$. La formula spiega il passo, ma non è ancora un comando: `fib` non è un'operazione del nostro linguaggio. Conserviamo allora il valore mancante in y. L'invariante diventa

$$I \equiv x = \text{fib}(i) \wedge y = \text{fib}(i-1) \wedge 1 \leq i \leq k.$$

Per avanzare vogliamo che il nuovo x valga il vecchio $x+y$ e che il nuovo y valga il vecchio x. Con $x=3, y=2$, i nuovi valori devono essere 5 e 3. La sequenza `x := x+y; y := x` produrrebbe invece 5 e 5. Salviamo prima il vecchio x:

```
x := 1; i := 1; y := 0;
while i != k do
    temp := x;
    x := x + y;
    y := temp;
    i := i + 1
endw
```



La coppia dei termini consecutivi si sposta di un posto. La variabile temporanea permette di usare il vecchio x anche dopo averlo aggiornato.

8.5.1 Controllare il passo e l'uscita

L'inizializzazione usa $\text{fib}(1)=1$ e $\text{fib}(0)=0$. Quindi vale l'invariante. Se siamo nel corpo, $1 \le i \le k$ e $i \neq k$ danno $i < k$.

Indichiamo con un apice i valori dopo il corpo. La somma produce il termine successivo; il temporaneo conserva quello che deve diventare il precedente:

$$x' = \text{fib}(i) + \text{fib}(i-1) = \text{fib}(i+1),$$

$$y' = \text{fib}(i), \qquad i' = i+1.$$

Riscrivendo rispetto al nuovo indice ritroviamo $x' = \text{fib}(i')$ e $y' = \text{fib}(i'-1)$. Inoltre $i < k$ assicura $1 \le i' \le k$: anche i limiti si conservano.

La quantità $k-i$ è non negativa e diminuisce di 1 a ogni giro. Tutte le operazioni sono definite, quindi il ciclo termina. All'uscita $i=k$ e l'invariante dà $x=\text{fib}(k)$.

Per $k=1$ la guardia è subito falsa e il risultato è già pronto. Il caso $k=0$ è escluso da questa specifica; la verifica finale chiede come includerlo.

8.5.2 Per proseguire: seguire una ricorrenza che dimezza l'indice

Un altro problema del corso mostra che far crescere un contatore non è sempre la scelta migliore. Definiamo $f(0)=0$, $f(1)=1$; per $j \ge 2$ poniamo $f(j)=f(j/2)$ se j è pari e $f(j)=f((j-1)/2)+f((j+1)/2)$ se è dispari. Gli argomenti a destra sono più piccoli di j, quindi la definizione permette di calcolare i valori a partire dai casi iniziali.

Per esempio $f(19)=f(9)+f(10)=f(4)+2f(5)=3f(2)+2f(3)=5f(1)+2f(2)=7$. In ogni passaggio compaiono due indici consecutivi. Cerchiamo allora di conservare

$$I \equiv 0 \le x \le n \wedge f(n) = k\,f(x) + h\,f(x+1),$$

con $n \geq 0$ non modificato. Inizializzando $x = n, k = 1, h = 0$ l'uguaglianza è vera. Quando $x = 0$ diventa $f(n) = k \cdot 0 + h \cdot 1 = h$: sappiamo già quale guardia scegliere e dove troveremo il risultato.

Deriviamo il corpo. Scriviamo $q = x \operatorname{div} 2$. Se $x = 2q > 0$, allora $f(x) = f(q)$ e $f(x+1) = f(q) + f(q+1)$; pertanto

$$k f(x) + h f(x+1) = (k+h) f(q) + h f(q+1).$$

Basta sommare h a k e dimezzare x. Se $x = 2q+1$, le identità danno invece

$$k f(x) + h f(x+1) = k f(q) + (k+h) f(q+1).$$

Basta sommare k a h e dimezzare x. Per $x = 1$, la prima identità del caso dispari è ancora vera perché $f(1) = f(0) + f(1)$.

Possiamo ora scrivere il programma, usando il test `x mod 2 = 0` per riconoscere gli indici pari:

```
x := n; k := 1; h := 0;
while x != 0 do
    if x mod 2 = 0
        then k := h + k
        else h := h + k
    fi;
    x := x div 2
endw
```

Prima aggiorniamo il coefficiente richiesto dal caso pari o dispari, poi dimezziamo l'indice. Il risultato sarà in h.

Le uguaglianze appena ottenute provano che l'invariante si conserva. Sotto la guardia $x > 0$, e $0 \leq x \operatorname{div} 2 < x$; dunque x è una funzione di terminazione. Il divisore 2 è sempre non nullo.

All'uscita $h = f(n)$, anche per $n = 0$, quando il corpo non viene eseguito. Le asserzioni guidano la costruzione; il programma usa solo somme, divisione intera e modulo.

Verifica 8

1. Vogliamo conservare $x\,y + p\,q = n$ mediante `x := x-p; q := E`. Trova un'espressione E che funzioni per tutti gli interi e giustifica anche il caso $p = 0$.

2. Nel programma di Fibonacci modifica soltanto l'inizializzazione e l'invariante per calcolare anche il caso $k = 0$. Puoi mantenere la stessa forma del corpo usando due termini consecutivi a indici non negativi? Indica quale variabile conterrà il risultato.

Array: indici, aggiornamenti e zone

In breve
Come lavoriamo su una sequenza senza sbagliare posizione? Sappiamo già usare contatori e invarianti. Negli array daremo un significato agli indici e ai confini tra la parte già trattata e quella ancora da esaminare.

Per sommare i voti di un elenco vogliamo conservare molti valori con un solo nome. Un **array** raccoglie la sequenza; un **indice** permette di scegliere una posizione. Dovremo controllare quali posizioni possiamo leggere e modificare.

9.1 Array come funzioni sugli indici

La dichiarazione `v : array [0,4) of int` descrive quattro interi con indici $0, 1, 2, 3$. L'estremo destro 4 segna la fine: non è una posizione leggibile. In generale un array omogeneo, cioè con elementi dello stesso tipo, è una funzione

$$v\colon [a, b) \to A,$$

con indici naturali $a \le b$ e $A = \mathbb{Z}$ oppure $A = \text{Bool}$. L'intervallo $[a, b)$ comprende gli indici $a \le i < b$, ha $b - a$ posizioni ed è vuoto se $a = b$. Scriviamo $\text{dom}(v) = [a, b)$ per il dominio degli indici validi.

Lo stato associa ora al nome v un'intera funzione $\sigma(v)$. Se $\sigma(v) = \{0 \mapsto 2, 1 \mapsto 1, 2 \mapsto 10, 3 \mapsto 6\}$, leggere `v[2]` restituisce 10. Si calcola prima l'espressione indice e poi si applica la funzione:

$$\mathcal{E}(v[E], \sigma) = \sigma(v)(\mathcal{E}(E, \sigma)).$$

Questa formula ha senso solo se il valore dell'indice esiste e appartiene al dominio.

Un indice può contenere altre letture: `v[v[0]+v[1]]` legge prima 2 e 1, somma ottenendo 3, poi legge `v[3]`, cioè 6. Dobbiamo controllare sia gli indici interni 0 e 1, sia l'indice esterno 3.

Se la somma fosse 4, gli accessi interni sarebbero leciti ma quello esterno fallirebbe.

Concetto chiave · Definitezza di un accesso
Per un'espressione indice intera E,

$$\text{def}(v[E]) \equiv \text{def}(E) \wedge E \in \text{dom}(v).$$

La correttezza richiede che ogni lettura effettivamente eseguita abbia un indice valido. Per un array su $[0, n)$, il solo vincolo $x \le n$ non autorizza a leggere `v[x]`: occorre $0 \le x < n$.

Le cause di indefinitezza comprendono ora divisori nulli e indici fuori dominio. I tipi restano un altro controllo: un array booleano produce booleani, che non possono essere usati come indici interi. Gli operatori del linguaggio agiscono sui singoli elementi: una scrittura come `v+w`, intesa come somma di interi array, non è ammessa.

9.2 Aggiornamento selettivo: prima valutare, poi modificare

Il comando `v[E] := E'` cambia una sola posizione. Tanto l'indice quanto il valore a destra si calcolano nello stato precedente. Per esempio, con l'array appena visto, `v[v[0]] := v[1]+5` calcola l'indice 2 e il valore 6, poi cambia soltanto `v[2]`. Il nuovo array è $\{0 \mapsto 2, 1 \mapsto 1, 2 \mapsto 6, 3 \mapsto 6\}$.

Per descrivere formalmente l'effetto, poniamo $j = \mathcal{E}(E, \sigma)$ e $d = \mathcal{E}(E', \sigma)$. Se entrambi sono definiti e $j \in \mathrm{dom}(v)$, costruiamo la nuova sequenza $w = \sigma(v)[d/j]$: essa vale d all'indice j e coincide con $\sigma(v)$ a ogni altro indice. Infine aggiorniamo lo stato:

$$\mathcal{C}(v[E] := E', \sigma) = \sigma[w/v].$$

L'aggiornamento interno riguarda una posizione della sequenza; quello esterno cambia il valore associato al nome v. Il dominio resta lo stesso.

9.2.1 La condizione prima di un aggiornamento

Per verificare una postcondizione P, scriviamo simbolicamente $w = v[E'/E]$. L'assioma di assegnamento richiede prima del comando

$$\mathrm{def}(E) \wedge \mathrm{def}(E') \wedge E \in \mathrm{dom}(v) \wedge P[w/v].$$

Dopo `v[E] := E'` vale allora P. Come per le variabili con un solo valore, una precondizione più forte può essere usata tramite la regola della conseguenza. Qui sostituiamo l'intera sequenza nell'asserzione: il nuovo elemento in posizione i è E' se $i = E$, e il vecchio $v[i]$ se $i \neq E$.

Esempio guidato · Estendere un prefisso di somme

Supponiamo $1 \le k < n$ e che, prima della posizione k, l'array contenga già le somme $a[i] = \sum_{j=0}^{i} j$ per ogni $0 \le i < k$. Eseguiamo `a[k] := a[k-1]+k`.

I vincoli su k rendono validi entrambi gli indici k e $k-1$. Per dimostrare la nuova proprietà fino a k incluso, separiamo due casi. Se $i < k$, la posizione i non viene toccata e conserva la somma corretta. Se $i = k$, il nuovo valore è

$$a[k-1] + k = \sum_{j=0}^{k-1} j + k = \sum_{j=0}^{k} j.$$

Abbiamo così verificato tutti gli indici $0 \le i \le k$. La distinzione tra posizione modificata e posizioni conservate è ciò che permette di provare una postcondizione quantificata.

9.3 Incrementare un array: un confine tra fatto e da fare

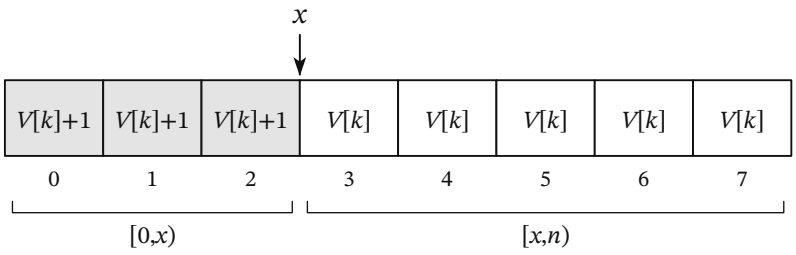
Vogliamo aggiungere 1 a ogni elemento di `a : array [0,n) of int`, con $n \ge 0$. Chiamiamo V la sequenza iniziale, una variabile di specifica che il programma non modifica. Il risultato richiesto è $\forall k \in [0, n).\, a[k] = V[k] + 1$.

Pensiamo a un confine x: a sinistra abbiamo già incrementato, da x in poi dobbiamo ancora farlo. All'inizio $x = 0$; alla fine vogliamo $x = n$. Da questa immagine ricaviamo

$$I \equiv 0 \le x \le n \wedge (\forall k \in [0, x).\, a[k] = V[k] + 1)$$

$$\wedge (\forall k \in [x, n).\, a[k] = V[k]).$$

La seconda riga non è superflua: per concludere che incrementare `a[x]` produce $V[x] + 1$, dobbiamo sapere che prima vale ancora $V[x]$.



Il confine x separa le posizioni già incrementate da quelle ancora uguali alla sequenza iniziale V. Il confine può anche essere 0 oppure n.

```
x := 0;
while x < n do
    a[x] := a[x] + 1;
    x := x + 1
endw
```

La precondizione è $n \geq 0 \wedge a = V$, dove l'uguaglianza indica che coincidono tutti gli elementi. Con $x = 0$, il prefisso elaborato è vuoto e la proprietà sul resto è esattamente quella iniziale: si stabilisce I.

Prima del corpo, $I \wedge x < n$ dà $0 \leq x < n$: la lettura e la scrittura sono lecite. Le posizioni prima di x restano incrementate, quella in x passa da $V[x]$ a $V[x]+1$, quelle dopo restano iniziali. Abbiamo quindi la stessa divisione con confine $x+1$. Il secondo assegnamento dà a x proprio quel valore, ristabilendo I.

9.3.1 Quando il confine raggiunge la fine

La quantità $n - x$ è non negativa e diminuisce di 1. Il ciclo termina e, quando la guardia è falsa, $x \geq n$; insieme a $x \leq n$ otteniamo $x = n$. L'intero array appartiene allora alla parte elaborata. Per $n = 0$ le proprietà quantificate riguardano l'insieme vuoto e sono soddisfatte senza eseguire il corpo.

9.4 Somma di un array: sostituire la fine con un confine mobile

Ora l'array non deve cambiare; vogliamo soltanto ottenere

$$s = \sum_{i=0}^{n-1} a[i].$$

Sostituiamo nella postcondizione il valore finale n con un indice di lavoro x: s sarà la somma delle prime x posizioni. Per $x = 0$ questa è la somma vuota, che vale 0. Per $x = n$ sarà la somma desiderata.

```
x := 0; s := 0;
while x != n do
    s := s + a[x];
    x := x + 1
endw
```

Usiamo $I \equiv s = \sum_{i=0}^{x-1} a[i] \wedge 0 \leq x \leq n$. L'uguaglianza spiega l'accumulatore; i limiti sul contatore permettono la verifica. Infatti, sotto la guardia $x \neq n$, essi danno $0 \leq x < n$, quindi `a[x]` è definito.

All'inizio I vale per la convenzione sulla somma vuota. Se vale prima del corpo, dopo l'aggiunta otteniamo

$$s' = \sum_{i=0}^{x-1} a[i] + a[x] = \sum_{i=0}^{x} a[i].$$

Poiché il nuovo contatore è $x' = x + 1$, questa è la somma fino a $x' - 1$. Inoltre $x < n$ implica $x' \le n$, dunque l'invariante è ristabilito. Ancora $n - x$ prova la terminazione; la guardia falsa dà direttamente $x = n$. Con $n = 0$ il risultato è 0.

Sostituire una costante della specifica con una variabile ha trasformato il risultato completo in una proprietà di risultati parziali. I vincoli aggiunti su x servono insieme al controllo degli indici, all'invarianza e alla terminazione.

9.5 Bandiera a due colori: classificare senza perdere elementi

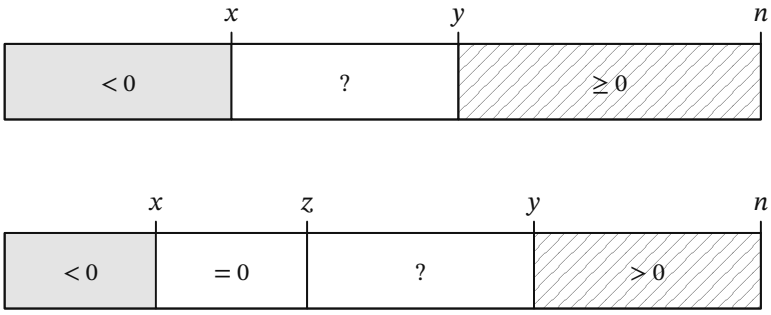
Vogliamo riordinare un array in modo che prima vengano i negativi e poi i valori positivi o nulli. Non chiediamo l'ordinamento interno a ciascun gruppo. Ma chiediamo di conservare tutti i valori, comprese le loro ripetizioni: scriviamo $\mathrm{perm}(a, A)$ se a ha gli stessi elementi della sequenza iniziale A, con le stesse molteplicità.

Un solo confine non basta a distinguere gli elementi già messi a destra da quelli ancora da guardare. Introduciamo due confini $x \le y$: il prefisso $[0, x)$ contiene negativi, il suffisso $[y, n)$ contiene non negativi, mentre $[x, y)$ è ancora sconosciuto. L'invariante è

$$I \equiv \mathrm{perm}(a, A) \wedge 0 \le x \le y \le n$$

$$\wedge (\forall i \in [0, x).\, a[i] < 0) \wedge (\forall i \in [y, n).\, a[i] \ge 0).$$

Non affermiamo nulla sui segni della zona sconosciuta: è proprio il lavoro che resta. La sua lunghezza $y - x$ suggerisce insieme la guardia `x != y` e la funzione di terminazione. Con $x = 0$, $y = n$ entrambe le zone classificate sono vuote.



In alto, due categorie e una zona ancora sconosciuta. In basso, lo stesso ragionamento è esteso a negativi, zeri e positivi: cambiano i confini, mentre resta centrale la zona da esaminare.

Sotto la guardia $x < y$. Possiamo leggere `a[x]` e, se serve, `a[y-1]`: entrambi gli indici sono compresi tra 0 e $n - 1$. Se $a[x] < 0$, abbiamo trovato un altro elemento per il prefisso: basta avanzare x. Se invece $a[x] \ge 0$, vogliamo portarlo a destra, nella posizione $y - 1$, e arretrare y.

Una copia `a[y-1] := a[x]` perderebbe il vecchio elemento di destra. Per conservare la permutazione facciamo uno scambio usando una variabile temporanea. Il valore che arriva in posizione x è ancora sconosciuto: in questo ramo x deve restare fermo.

```
x := 0; y := n;
while x != y do
    if a[x] < 0
        then x := x + 1
        else temp := a[y-1];
             a[y-1] := a[x];
             a[x] := temp;
             y := y - 1
    fi
endw
```

9.5.1 Perché x resta fermo dopo lo scambio

Partiamo da quattro elementi $a = [4, -2, 0, 7]$, con $x = 0, y = 4$. Il primo elemento è non negativo: lo scambiamo con l'ultimo e arretriamo y.

Controllo	a	x	y
iniziale	$[4, -2, 0, 7]$	0	4
dopo il primo giro	$[7, -2, 0, 4]$	0	3
dopo il secondo giro	$[0, -2, 7, 4]$	0	2

Dopo il primo giro è arrivato 7 in posizione 0. Non era stato esaminato: occorre controllarlo nel secondo giro, che lo sposta a destra. Anche il nuovo 0 dovrà essere esaminato al giro seguente.

Se dopo il primo scambio avessimo aumentato x, avremmo dichiarato negativo il valore 7. Si perderebbe subito il significato del prefisso. Per questo lo scambio fa avanzare soltanto il confine destro.

9.5.2 Conservare le zone e arrivare all'uscita

Verifichiamo i due rami. Nel primo l'array non cambia e il solo elemento aggiunto al prefisso è negativo.

Nel secondo, lo scambio non modifica né il vecchio prefisso $[0, x)$ né il vecchio suffisso $[y, n)$; mette un non negativo in $y - 1$ e conserva le molteplicità. Dopo l'arretramento, il nuovo suffisso è quindi tutto non negativo.

Se $x = y - 1$, lo scambio riguarda una posizione con sé stessa: funziona ugualmente.

Ogni ramo diminuisce $y - x$ di 1, mantenendo $0 \le x \le y \le n$. Il ciclo termina. All'uscita $x = y$, quindi la zona sconosciuta è vuota: il prefisso è negativo e l'intero resto è non negativo.

La postcondizione comprende $0 \le x \le n$, con entrambi gli estremi ammessi. Se tutti i valori sono negativi, $x = n$; se nessuno lo è, $x = 0$. Anche l'array vuoto è ammesso.

9.5.3 Per proseguire: bandiera a tre colori

Separiamo ora negativi, zeri e positivi. Per una classificazione completa servono due confini finali; durante il lavoro ne usiamo tre:

$$0 \le x \le z \le y \le n.$$

Manteniamo $\text{perm}(a, A)$, negativi in $[0, x)$, zeri in $[x, z)$, positivi in $[y, n)$. La parte sconosciuta è $[z, y)$. Inizializziamo $x = z = 0, y = n$; continuiamo finché $z \neq y$ e misuriamo il lavoro con $y - z$.

Se $a[z] < 0$, dobbiamo portarlo subito dopo il prefisso negativo, cioè in x. Scambiamo $a[x]$ e $a[z]$, poi avanziamo entrambi x e z.

Quando $x < z$, il vecchio $a[x]$ era uno zero, quindi può diventare l'ultimo elemento della nuova zona degli zeri.

Quando $x = z$, lo scambio è con sé stesso e i confini avanzano insieme senza creare zeri inesistenti.

9.5.4 Gli altri due casi

Se $a[z] = 0$, basta avanzare z. Se $a[z] > 0$, arretriamo y, scambiamo l'elemento positivo con quello della nuova posizione y e lasciamo z fermo: occorre esaminare ciò che è arrivato da destra.

```
x := 0; z := 0; y := n;
while z != y do
    if a[z] < 0
        then temp := a[x]; a[x] := a[z]; a[z] := temp;
             x := x + 1; z := z + 1
        else if a[z] = 0
                 then z := z + 1
                 else y := y - 1;
                      temp := a[y]; a[y] := a[z]; a[z] := temp
             fi
    fi
endw
```

9.5.5 Indici, progresso e risultato

Sotto la guardia $z < y$, tutti gli accessi sono validi. In particolare, nel ramo positivo il nuovo y è il vecchio $y - 1 \geq z$.

Gli scambi conservano le molteplicità; l'analisi dei tre casi mostra che ciascuna zona mantiene il proprio significato. Ogni ramo diminuisce $y - z$ di 1, quindi il ciclo termina.

All'uscita $z = y$: la zona sconosciuta scompare e restano, nell'ordine, negativi, zeri e positivi. Questo sviluppo completa l'esercizio proposto nelle lezioni.

Verifica 9

1. Per `v : array [0,4) of int` con valori $[2, 1, 10, 6]$, esegui `v[v[0]] := v[0]+v[1]`. Indica tutti gli indici controllati e il nuovo array. Spiega perché il comando successivo `v[v[2]] := 0` è definito oppure non lo è.

2. Modifica il programma della somma per contare gli elementi negativi di un array. Scrivi un invariante che dia significato al contatore, controlla gli indici e giustifica terminazione e risultato anche per l'array vuoto.

Soluzioni ragionate

A.1 Capitolo 1 — Specifiche e stati

1.1. Prodotto per somme. Ai controlli della guardia gli stati sono $(A, B, C) = (4, 2, 0)$, poi $(4, 1, 4)$ e infine $(4, 0, 8)$. C raccoglie la parte già calcolata; B conta le aggiunte rimaste.

Il valore iniziale era $b = 2$, ma alla fine $B = 0$. Quindi il prodotto corrente $A\,B = 0$ non ricorda quello iniziale. La richiesta è $C = a\,b = 8$.

1.2. Il maggiore. Chiamiamo x, y gli ingressi e r il risultato. Le lettere di specifica a, b ricordano i valori iniziali. La precondizione è $x = a \land y = b$, con $a, b \in \mathbb{Z}$.

Alla fine vogliamo conservare $x = a \land y = b$ e ottenere

$$r \geq a \land r \geq b \land (r = a \lor r = b).$$

Le disuguaglianze da sole ammetterebbero numeri troppo grandi. L'ultima condizione richiede che r sia uno dei due ingressi: insieme, le tre condizioni dicono che è il massimo.

La specifica dice che cosa ottenere, ma non sceglie ancora il confronto e gli assegnamenti che lo produrranno.

A.2 Capitolo 2 — Calcolo proposizionale

2.1. Uscire senza pioggia. «Esco solo se non piove» è $e \Rightarrow \neg p$: uscire richiede che non piova. «Se non piove, esco» è invece $\neg p \Rightarrow e$: non piovere basterebbe per uscire.

Scegliamo p falsa ed e falsa: non piove e resto a casa. La prima formula è vera, la seconda falsa. Questo controesempio mostra che la prima non implica la seconda.

2.2. Semplificare e seguire i segni. Distribuiamo per avvicinare P e la sua negazione:

$$P \land (\neg P \lor Q) \equiv (P \land \neg P) \lor (P \land Q) \equiv F \lor (P \land Q) \equiv P \land Q.$$

I passi usano distributività, contraddizione e unità.

In $\neg(P \Rightarrow Q) \land R$, il percorso verso P attraversa la negazione e l'antecedente: due inversioni, quindi segno positivo. Per Q attraversiamo soltanto la negazione: segno negativo. Per R non ci sono inversioni: segno positivo.

La riscrittura $(P \land \neg Q) \land R$ conferma i tre segni.

A.3 Capitolo 3 — Individui e formule

3.1. Sostituire senza catturare. In $(\forall x.p(x, y)) \land (\exists y.q(x, y))$, la x dentro p è legata e la y è libera. Dentro q accade il contrario: x è libera e y legata.

Vogliamo sostituire y alla sola x libera. Prima rinominiamo il quantificatore esistenziale in $\exists z$: altrimenti la y inserita verrebbe catturata. Poi sostituiamo e otteniamo

$$(\forall x.p(x, y)) \land (\exists z.q(y, z)).$$

La prima parte resta invariata, perché non contiene una x libera. Le due y rimaste sono libere: $\exists z$ non le lega.

3.2. Una proprietà generale e un unico testimone. Con $s(n) = n + 1$ scriviamo

$$\forall x.(\text{pari}(x) \Rightarrow \text{pari}(s(s(x)))).$$

$s(s(x))$ indica un numero ed è un termine; l'intera scrittura esprime una proprietà ed è una formula chiusa.

Per dire «esattamente un elemento soddisfa p» usiamo

$$\exists x.(p(x) \wedge \forall y.(p(y) \Rightarrow y = x)).$$

La parte $p(x)$ garantisce un testimone. L'universale dice che ogni altro testimone coincide con esso: l'uguaglianza garantisce l'unicità.

A.4 Capitolo 4 — Semantica e quantificatori

4.1. Trovare un controesempio. Neghiamo prima l'universale, poi l'implicazione:

$$\neg(\forall x.(p(x) \Rightarrow \exists y.q(x, y))) \equiv \exists x.(p(x) \wedge \neg(\exists y.q(x, y))).$$

Un secondo De Morgan porta la negazione fino agli atomi:

$$\exists x.(p(x) \wedge \forall y.\neg q(x, y)).$$

Dobbiamo trovare un individuo con proprietà p per cui la relazione q fallisca con tutti gli individui y. Non basta una sola coppia per cui q è falsa.

4.2. Cambiare il testimone o sceglierne uno solo. $\forall x.\exists y.R(x, y)$ è vera: per $x = a$ scegliamo $y = b$; per $x = b$ scegliamo $y = a$.

$\exists y.\forall x.R(x, y)$ è falsa. Il candidato $y = a$ fallisce per $x = a$; il candidato $y = b$ fallisce per $x = b$. Ogni elemento ha un compagno, ma nessuno è compagno di tutti.

Per la generalizzazione scegliamo sui naturali $P(x)$: «x è pari» e una costante d che indica 2. $P(d)$ è vera, ma $\forall x.P(x)$ è falsa per 1. Quel nome aveva una proprietà particolare: non rappresentava un individuo generico.

A.5 Capitolo 5 — Stati, espressioni e comandi

5.1. Scambiare con somme e differenze. Da $(x, y) = (3, 7)$ i tre comandi producono, nell'ordine, $(10, 7)$, $(10, 3)$ e $(7, 3)$.

Usiamo ora gli ingressi generici A, B. Dopo il primo comando abbiamo $(A + B, B)$. Il secondo legge il nuovo x e calcola $(A + B) - B = A$: otteniamo $(A + B, A)$.

Il terzo calcola $(A + B) - A = B$, quindi lo stato finale è (B, A). Abbiamo provato lo scambio per tutti gli interi matematici. Con interi rappresentati da un numero limitato di cifre dovremmo controllare anche se la somma è rappresentabile.

5.2. Diminuire non basta. Da 4 otteniamo $4, 2, 0$: dopo due giri la guardia è falsa. Da 5 otteniamo invece $5, 3, 1, -1, -3, \ldots$.

Sottrarre 2 conserva la parità: nel secondo caso ogni valore rimane dispari e non può essere zero. La guardia resta vera e il ciclo non termina. Una variabile può diminuire senza raggiungere il valore richiesto per uscire.

A.6 Capitolo 6 — Triple di Hoare e invarianti

6.1. Risalire attraverso un assegnamento. Il valore finale di y sarà $2x$. Sostituendolo nella postcondizione $y > 10$ otteniamo $2x > 10$, cioè $x > 5$.

La moltiplicazione è sempre definita. Poiché x è intero, la condizione equivale a $x \geq 6$. L'ipotesi proposta basta dunque per l'assioma dell'assegnamento, su tutti gli ingressi ammessi.

6.2. Una divisione che fallisce sempre. Prima di `y := z div x` serve $x \neq 0$ per la definitezza, oltre alla postcondizione $z > 4$, che l'assegnamento non cambia.

Risalendo attraverso `x := 0`, sostituiamo 0 a x e otteniamo

$$0 \neq 0 \wedge z > 4.$$

La richiesta è falsa in ogni stato: nessun ingresso può rendere questa esecuzione corretta in senso totale. La precondizione data è invece soddisfatta da uno stato con $z = 5, x = 3$: da quello stato il comando tenta comunque di dividere per zero.

A.7 Capitolo 7 — Correttezza totale

7.1. Un contatore che raggiunge n. Scegliamo $t = n - i$. Dall'invariante segue $i \leq n$, quindi $t \geq 0$.

Se prima del corpo $n - i = V$, dopo l'incremento vale $n - (i + 1) = V - 1 < V$. Inoltre $i < n$ e il fatto che i valori siano interi danno $i + 1 \leq n$: l'invariante si conserva.

Guardia e corpo sono sempre definiti e il singolo incremento termina. Il ciclo termina; all'uscita $i \geq n$ e l'invariante $i \leq n$ danno $i = n$.

7.2. Aggiungere dopo l'incremento. Ora il corpo aumenta prima x e poi aggiunge il valore nuovo. L'invariante adatto è

$$I \equiv s = \sum_{j=1}^{x} j \wedge 0 \leq x \leq n.$$

All'inizio la somma è vuota e vale zero. Se prima $x = k$, il corpo porta x a $k + 1$ e aggiunge proprio $k + 1$ a una somma che arriva a k. Si ristabilisce la stessa proprietà con il nuovo contatore.

La funzione di terminazione resta $n - x$. All'uscita $x = n$, quindi abbiamo sommato da 1 a n. Per $n = 0$ la somma è vuota: il risultato è zero senza eseguire il corpo.

A.8 Capitolo 8 — Costruire un ciclo

8.1. Compensare una modifica. Dopo `x := x-p`, il termine $x\,y$ ha perso $p\,y$. Possiamo compensare aumentando q di y, così $p\,q$ guadagna proprio $p\,y$. Scegliamo $E = q + y$.

In termini dei valori precedenti ai due comandi:

$$(x - p)y + p(q + y) = x\,y - p\,y + p\,q + p\,y = x\,y + p\,q = n.$$

L'espressione $q + y$ non legge x, quindi il primo comando non la cambia. Per $p = 0$ la conservazione non impone un nuovo valore particolare a q, ma la scelta funziona comunque. Tutte le espressioni sono definite sugli interi.

8.2. Fibonacci includendo zero. Mettiamo il risultato in `y`. Inizializziamo `x := 1; i := 0; y := 0` e scegliamo

$$I \equiv x = \mathrm{fib}(i + 1) \wedge y = \mathrm{fib}(i) \wedge 0 \leq i \leq k.$$

Ora entrambi gli indici della successione sono non negativi, anche all'inizio. Il corpo resta `temp := x; x := x+y; y := temp; i := i+1`.

Dopo gli assegnamenti, x contiene $\mathrm{fib}(i + 2)$ e y contiene $\mathrm{fib}(i + 1)$, dove i indica il vecchio contatore. Con il nuovo indice $i' = i + 1$ ritroviamo esattamente l'invariante.

Sotto la guardia $i \neq k$, i limiti danno $i < k$, quindi l'incremento non supera k. La funzione $k - i$ prova la terminazione. All'uscita $i = k$ e $y = \mathrm{fib}(k)$.

Per $k = 0$ la guardia è subito falsa e $y = 0$: anche questo caso è incluso.

A.9 Capitolo 9 — Array

9.1. Valutare prima di aggiornare. Nell'indice, `v[0]` vale 2. A destra leggiamo le posizioni 0 e 1 e otteniamo $2 + 1 = 3$. Gli indici usati sono 0, 1 e 2, tutti validi.

Cambiamo soltanto la posizione 2: il nuovo array è $[2, 1, 3, 6]$. Il comando successivo legge `v[2]`, ora uguale a 3, e scrive in posizione 3. È quindi definito e lascia $[2, 1, 3, 0]$.

Se eseguissimo il secondo comando per primo, leggerebbe 10 in posizione 2 e tenterebbe di scrivere fuori dall'array. L'ordine dei comandi può cambiare anche la definitezza degli accessi.

9.2. Contare i negativi. Usiamo c per il numero di negativi incontrati. Il programma è:

```
x := 0; c := 0;
while x != n do
    if a[x] < 0 then c := c + 1 else skip fi;
    x := x + 1
endw
```

L'invariante richiede $0 \le x \le n$ e dice che c conta gli indici $i \in [0, x)$ con $a[i] < 0$. All'inizio il prefisso è vuoto e $c = 0$.

Sotto la guardia abbiamo $0 \le x < n$, quindi la lettura è valida. Se il nuovo elemento è negativo, aumentiamo c di uno; altrimenti lo lasciamo invariato. Dopo l'incremento di x, il prefisso comprende esattamente una posizione in più: quella appena esaminata.

La quantità $n - x$ è non negativa e diminuisce di 1. Il corpo termina con successo, quindi il ciclo termina. All'uscita $x = n$ e c conta i negativi dell'intero array.

Se $n = 0$, il risultato è già $c = 0$ e non si legge alcuna posizione.